Bachelor's Thesis Computing Science




Radboud University Nijmegen

---

**Formalising Modal Embeddings of Call-by-Name and Call-by-Value**

---

*Author:*
Floris Reuvers
s1096976

*First supervisor/assessor:*
Dr. Niels van der Weide

*Second assessor:*
Dr. Engelbert Hubbers

June 16, 2025

**Abstract**

This thesis presents a formalisation of the boxed $\lambda$-calculus ($\lambda_b$) in the formal proof assistant Agda. This calculus is a refinement of $\lambda_\Box$, a modal $\lambda$-calculus that is a fragment of IS4 [23]. Girard's and Gödel's embeddings are translations from the call-by-name $\lambda$-calculus ($\lambda_n$) and the call-by-value $\lambda$-calculus ($\lambda_v$) into the shared target, call-by-box $\lambda$-calculus ($\lambda_b$). We implement these embedding in Agda. We then proof that Girard's embedding preserves reduction and evaluation, and that Gödel's embedding preserves reduction. Preservation of Gödel's embedding is not proven in Agda, due to time limitations, but it is shown in [23].

# Contents


**1 Introduction** **3**

**2 Lambda Calculus** **5**
2.1 Introduction to the $\boldsymbol{\lambda}$-calculus . . . 5
2.2 Well-Typedness . . . 6
2.3 $\boldsymbol{\beta}$-Reduction . . . 7
2.3.1 Free and Bound Variables . . . 7
2.3.2 Substitution . . . 8
2.3.3 $\boldsymbol{\alpha}$-equivalence . . . 9
2.3.4 $\boldsymbol{\beta}$-Reduction . . . 10
2.4 Call-by-name $\boldsymbol{\lambda}$-calculus . . . 11
2.5 Call-by-value $\boldsymbol{\lambda}$-calculus . . . 12

**3 Call-by-box Lambda Calculus** **14**
3.1 Specification of $\boldsymbol{\lambda_b}$ . . . 14
3.1.1 Intuitionistic logic IS4 . . . 14
3.1.2 Terms and Types of $\boldsymbol{\lambda_b}$ . . . 15
3.2 Substitution and Free and Bound variables in $\lambda_b$ . . . 16

**4 Embeddings in $\boldsymbol{\lambda_b}$** **20**
4.1 Girard's embedding . . . 20
4.2 Gödel's Embedding . . . 22

**5 Formalisation in Agda** **27**
5.1 Intro to Agda . . . 27
5.1.1 Data Types and Functions . . . 27
5.1.2 Hole-Based Programming . . . 29
5.2 Challenges of the Formalisation . . . 31
5.2.1 De Bruijn Indices . . . 31
5.2.2 Well-typed Terms . . . 32
5.2.3 Substitution . . . 36
5.2.4 Weakening and Renaming . . . 39
5.3 Overview of the Formalisation . . . 40
5.3.1 File Structure . . . 40

5.3.2 Helper Lemmas . . . . . 41
5.3.3 Proofs of the Main Propositions . . . . . 43

**6 Related Work** **47**

**7 Conclusions and Future Work** **48**
7.1 Conclusions . . . . . 48
7.2 Future Work . . . . . 49

**A Appendix** **53**

# Chapter 1

# Introduction

One of the simplest, yet most powerful, models of computation is the $\lambda$-calculus. It serves as the foundation of many programming languages and proof assistants, such as Haskell and Agda. The two main well-known evaluation strategies for the $\lambda$-calculus are *call-by-name* (cbn) and *call-by-value* (cbv). Call-by-name does not evaluate an argument to a value until it is needed, while call-by-value evaluates an argument to a value before passing it to a function. These are two very different approaches and depending on the context, one may be more suitable than the other.

Cbn is more efficient when some arguments are not used inside a function, as it avoids unnecessary computation. However, if an argument is used multiple times, cbv is preferable, as it avoids recomputing the argument each time it is used. Therefore, a $\lambda$-calculus that can express both cbn and cbv is ideal. It allows for the creation of a programming language or proof assistant that is more flexible in evaluation strategies. While cbv can be simulated in Haskell, it requires some workarounds and support for cbv is not built into the language.

Many approaches of unifying cbn and cbv have been proposed. Thomas Ehrhard and Giulio Guerrieri [13] proposed a linear logic approach, just like the one by Maraist et al. [19], while Hatcliff, John and Danny, Olivier [15] proposed a thunk-based approach. However, we focus on the modal approach, introduced by Espírito Santo et al. [23]. This approach is based on the modal $\lambda$-calculus $\lambda_{\Box}$, which is a fragment of Bierman and de Paiva's intuitionistic modal logic (IS4) [7]. The proposed call-by-box $\lambda$-calculus ($\lambda_b$) is a refinement of $\lambda_{\Box}$, which has simpler typing rules and a more straightforward reduction strategy. This thesis aims to formalise this approach in Agda, which is a dependently typed programming language and proof assistant. Although $\lambda_b$ is discussed in the literature, it has not yet been formalised in a proof assistant before. Given the extra complexity of the $\lambda_b$ in comparison to the standard $\lambda$-calculus, it is important to formalise it to ensure that the properties of $\lambda_b$ are correct and machine-checked. Therefore,

the main research question of this thesis is as follows:

> How can the unification of call-by-name and call-by-value using modal logic be formalised in Agda?

In $\lambda_b$, cbn and cbv can be simulated by using *Girard's* and *Gödel's* embeddings, respectively. These are embeddings from the standard $\lambda$-calculus to $\lambda_b$, preserving both reduction and evaluation. This thesis formalises both embeddings in Agda and we prove the following:

- Girard's embedding preserves reduction.
- Girard's embedding preserves evaluation.
- Gödel's embedding preserves reduction.

We did not formally prove that Gödel's embedding preserves evaluation, but a less formal proof is given by [23]

We now give a brief overview of the chapters in this thesis. Chapter 2 introduces the $\lambda$-calculus, with substitution, $\beta$-reduction and evaluation relations. The next chapter, chapter 3, introduces $\lambda_b$, with substitution, $\beta$-reduction and its evaluation relations. We then discuss the two embeddings in chapter 4, where we also provide the propositions about the embeddings. Chapter 5 explains the basic Agda concepts, the challenges we faced while formalising $\lambda_b$ and their embeddings, and how we solved them. Finally, chapter 7 concludes the thesis and chapter 6 gives an overview of related work in the field.

# Chapter 2

# Lambda Calculus

This section provides a detailed and formal description of the $\lambda$-calculus, defined by Alonzo Church in [8]. We define a formal grammar of the $\lambda$-calculus in the same way as [23], and give examples of some terms in the $\lambda$-calculus. We mostly follow the notation of $\lambda$-terms in [17]. Only for substitution, we use a different notation, which we discuss later in section 2.3.2. Then we explain $\beta$-reduction, the only way to reduce $\lambda$-terms [8], after which we define call-by-name evaluation and call-by-value evaluation [22].

## 2.1 Introduction to the λ-calculus

The grammar of the lambda calculus is as follows:

**Definition 2.1** ($\lambda$-calculus)**.** The terms of the $\lambda$-calculus are given by:

$$M, N, P, Q \quad ::= \quad x \mid \lambda x.M \mid MN$$

Terms are denoted by $M, N, P$ or $Q$ and can either be of the form $x$, $\lambda x.M$ or $MN$:

- $x$ is a variable, which is a symbol that represents an input.
- $\lambda x.M$ is an abstraction. An abstraction is an anonymous function, where $x$ is the parameter and $M$ is the body of the function.
- $MN$ is a function application, where $M$ and $N$ are terms.

The $\lambda$-term $\lambda x.x$ is an abstraction. This specific abstraction is called the identity function and has one input parameter, namely $x$, and returns the input $x$. The body of the function is also $x$, and the function is usually abbreviated as **I** as done in [10]. The $\lambda$-term $(\lambda x.x)y$ is a function application. So the identity function is applied to the variable $y$.

## 2.2 Well-Typedness

This section discusses the concept of well-typedness in the context of lambda calculus. We first discuss types, typing judgements and typing rules. We then explore how to use these rules to show that a $\lambda$-term is well-typed.

Types of $\lambda$-calculus are defined in Definition 2.2, and are either $X$, which is the base type, or a function type, $A \to A'$ with $A$ and $A'$ being types in $\lambda$-calculus. This formulation follows from Church's *simple typed lambda calculus*, introduced in [9]. In this thesis, we use the Curry-style notation where we write terms without explicit types. This notation is formulated by Haskell B. Curry, and the notation used here is discussed in [5].

**Definition 2.2. Types of $\lambda$-calculus**

$$A \quad ::= \quad X \mid \ A \to A'$$

A typing judgement is of the form $\Gamma \vdash M : A$, and it means that we can derive that $M$ has type $A$ from the assumptions in context $\Gamma$. A context $\Gamma$ is a list of variable-type pairs, where each variable is associated with a type. A context is written as $\Gamma = x_1 : A_1, \ldots, x_n : A_n$, where each $x_i$ is a variable and each $A_i$ is a type.

The typing rules are defined in Definition 2.3, and are from [5]. The Id rule states that if $x$ has type $A$ in the context, we can conclude that $x$ has type $A$. The $\to$-I rule states that if we have a term $M$ of type $A'$ in the context $\Gamma, x : A$, we can conclude that from context $\Gamma$ it follows that $\lambda x.M$ has type $A \to A'$. The $\to$-E rule states that if we have a term $M$ of type $A \to A'$ and a term $N$ of type $A$, we can conclude that the application $MN$ has type $A'$.

**Definition 2.3. Typing rules of $\lambda$-calculus**

$$\frac{}{\Gamma, x : A \vdash x : A}\ \text{Id}$$

$$\frac{\Gamma, x : A \vdash M : A'}{\Gamma \vdash \lambda x.M : A \to A'}\ \to\text{-I}$$

$$\frac{\Gamma \vdash M : A \to A' \qquad \Gamma \vdash N : A}{\Gamma \vdash MN : A'}\ \to\text{-E}$$

A $\lambda$-term is considered to be *well-typed* if we can derive a typing judgement with the typing rules. The $\lambda$-terms that are well-typed can be assigned a type. In Example 2.4, we show that $\lambda$-term $(\mathbf{I}(\lambda x.\mathbf{I}x))$ is well-typed by giving the full typing derivation.

**Example 2.4. Well-typedness of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$**

$$\dfrac{\dfrac{\dfrac{}{y : A \to A \vdash y : (A \to A)}\,\text{Id}}{\vdash \lambda y.y : (A \to A) \to (A \to A)}\,{\to}\text{-I} \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{x : A, z : A \vdash z : A}\,\text{Id}}{x : A \vdash \lambda z.z : A \to A}\,{\to}\text{-I} \quad \dfrac{}{x : A \vdash x : A}\,\text{Id}}{x : A \vdash \mathbf{I}x : A \to A}\,{\to}\text{-E}}{\vdash \lambda x.\mathbf{I}x : A \to A}\,{\to}\text{-I}}{\vdash \mathbf{I}(\lambda x.\mathbf{I}x) : A \to A}\,{\to}\text{-E}$$

In contrast to the $\lambda$-term $(\mathbf{I}(\lambda x.\mathbf{I}x))$, the lambda term $MM$ cannot be assigned a type. Since $MM$ is an application, we can only apply the $\to$-E rule, which would give a contradiction. The $\lambda$-term $M$ cannot be both a function and the argument to the same function.

## 2.3 $\beta$-Reduction

To include a notion of computation to the $\lambda$-calculus, we discuss $\beta$-reduction. This notion of computation was introduced by Alonzo Church in [8] and the definition of $\beta$-reduction we use is by [5]. This reduction relation is the only way to simplify terms in the $\lambda$-calculus. So with $\beta$-reduction, we can apply an abstraction to an argument. To do this, we give definitions of substitution, free and bound variables similar to the definitions in [5].

### 2.3.1 Free and Bound Variables

Let us first define free and bound variables [22, 5]. Let $P$ be any term in the $\lambda$-calculus, $FV(P)$ be the set that contains all free variables in $P$ and $FB$ be the set that contains all bound variables in $P$. We define both $FV$ and $BV$ inductively on $P$ by Definition 2.5 and Definition 2.6. Since $P$ can only be a variable, an abstraction or a function application, $P$ can only be of the form $x$, $\lambda x.M$ and $MN$. Therefore, we define $FV$ and $BV$ for $x$, $\lambda x.M$ and $MN$.

**Definition 2.5. FB**

$$\begin{aligned} FV(x) &= \{x\} \\ FV(\lambda x.M) &= FV(M) \backslash \{x\} \\ FV(MN) &= FV(M) \cup FV(N) \end{aligned}$$

Let us now explain $FV$. The variable $x$ in the $\lambda$-term $x$ is free. All free variables in an abstraction are those that are not the parameter of the abstraction. For an application $MN$, all free variables are all free variables of $\lambda$-term $M$ and $\lambda$-term $N$.

**Definition 2.6. BV**

$$\begin{aligned} BV(x) &= \emptyset \\ BV(\lambda x.M) &= BV(M) \cup \{x\} \\ BV(MN) &= BV(M) \cup BV(N) \end{aligned}$$

Let us now explain $BV$. The variable $x$ in the $\lambda$-term $x$ is a free variable, and the set of bound variables is empty. An abstraction $\lambda x.M$ binds the variable $x$. So the set of bound variables in an abstraction is the set of bound variables in $M$ plus the variable $x$. For an application $MN$, the bound variables are all bound variables of both $\lambda$-term $M$ and $\lambda$-term $N$.

We give two examples to illustrate the definitions of free and bound variables. We show that $\lambda$-term $\lambda x.yx$ has one free variable, $y$ and one bound variable $x$.

**Example 2.7. FV**

$$\begin{aligned} FV(\lambda x.yx) &= FV(yx)\backslash\{x\} \\ &= (FV(x) \cup FV(y))\backslash\{x\} \\ &= (\{x\} \cup \{y\})\backslash\{x\} \\ &= \{x, y\}\backslash\{x\} \\ &= \{y\} \end{aligned}$$

**Example 2.8. BV**

$$\begin{aligned} BV(\lambda x.xy) &= BV(xy) \cup \{x\} \\ &= (BV(x) \cup BV(y)) \cup \{x\} \\ &= (\emptyset \cup \emptyset) \cup \{x\} \\ &= \{x\} \end{aligned}$$

The following example will demonstrate that using two different variables with the same name in a $\lambda$-term is quite tricky: $(\lambda x.x)x$. By the definitions of $FV$ and $BV$, we can reason that $FV((\lambda x.x)x) = \{x\}$ and $BV((\lambda x.x)x) = \{x\}$. While these sets are the same, they do not refer to the same $x$. This is quite confusing, and we prefer to write $(\lambda y.y)x$ instead, where $y$ is a new "fresh" variable with a different name. We now have $FV((\lambda y.y)x) = \{x\}$ and $BV((\lambda y.y)x) = \{y\}$. These $\lambda$-terms are equivalent by the notion of $\alpha$-equivalence, which states that $\lambda$-terms that differ only by the names of bound variables are considered the same. We discuss this in more detail after we define substitution.

### 2.3.2 Substitution

Keeping the definitions of free and bound variables in mind, we now define substitution. In this thesis, substitution is denoted as $P[x \mapsto Q]$ and is

defined inductively on $P$ (see Definition 2.9). Essentially, it replaces all free occurrences of the variable $x$ in the $\lambda$-term $P$ by $\lambda$-term $Q$. We chose not to use the original notation of substitution by Curry, Feys, and Craig in [11], which is $[Q/x]P$, as we think the notation $P[x \mapsto Q]$ is more clear and intuitive. We adopt the definition of substitution from [22], defined by Plotkin.

**Definition 2.9. Substitution**

$x[x \mapsto Q] = Q$
$y[x \mapsto Q] = y$ if $(x \neq y)$
$(MN)[x \mapsto Q] = M[x \mapsto Q]N[x \mapsto Q]$
$(\lambda x.M)[x \mapsto Q] = \lambda x.M$
$(\lambda y.M)[x \mapsto Q] = \lambda z.M[y \mapsto z][x \mapsto Q]$ if $(x \neq y)$

with $z$ a variable defined by:

1. If $x \notin FV(N)$ or $y \notin FV(M)$ then $z = y$
2. Otherwise, $z$ can be any variable such that $z \notin FV(N)$ or $z \notin FV(M)$

The variable $z$ always exists. Choosing a fresh variable for $z$ always satisfies condition two. It might not be clear why we need to use the variable $z$ and one might expect the rule in Definition 2.10 is sufficient.

**Definition 2.10. Incorrect Substitution Rule**

$$(\lambda y.M)[x \mapsto Q] = \lambda z.M[x \mapsto Q] \text{ if } (x \neq y)$$

However, the following example illustrates why the more complicated rule is necessary. Consider the substitution $(\lambda y.xy)[x \mapsto y]$. The free variable of $\lambda$-term $\lambda y.xy$ is $x$ and the bound variable is $y$. If we use the simple rule above, we get that $(\lambda y.xy)[x \mapsto y] = \lambda y.yy$, the meaning of the $\lambda$-term changes. However, the free variable we substituted, is not free anymore. The variable $y$ is not a free variable in the term that we substituted $y$ in. Therefore, we change the bound variable $y$ in $\lambda y.xy$ to a new variable $z$. We can pick $z$ as $z$ is not a free variable in $xy$. Following the more complex rule, we get:

$$\begin{aligned}(\lambda y.xy)[x \mapsto y] &= \lambda z.(xy)[y \mapsto z][x \mapsto y] \\ &= \lambda z.yz\end{aligned}$$

### 2.3.3 $\alpha$-equivalence

Here, we define $\alpha$-equivalence. We use the definition by Plotkin in [22]. The idea is that renaming bound variables does not change the meaning of a

$\lambda$-term. For example, the $\lambda$-terms $\lambda x.x$ and $\lambda y.y$ are equivalent ($\lambda x.x \equiv_\alpha \lambda y.y$), since renaming all bound variables can result in identical $\lambda$-terms. The $\lambda$-terms $\lambda x.x$ and $\lambda y.y$ represent the same idea. We define the relation $P \equiv_\alpha P'$ inductively on $P$ and $P'$ in Definition 2.11.

**Definition 2.11. $\alpha$-equivalence**

$$\begin{array}{ll} x \equiv_\alpha y & \text{if } x = y \\ MN \equiv_\alpha M'N' & \text{if } M \equiv_\alpha M' \text{ and } N \equiv_\alpha N' \\ \lambda x.M \equiv_\alpha \lambda y.M' & \text{if } M \equiv_\alpha M'[y \mapsto x] \text{ with } x = y \text{ or } x \notin FV(M') \end{array}$$

The last rule states that abstractions $\lambda x.M$ and $\lambda y.M'$ are equivalent if $M$ and $M'$ are equivalent after substituting $y$ for $x$ in $M'$. However, $x$ should be equal to $y$ or $x$ should not be a free variable in $M'$.

**Example 2.12.** Consider the following:

- $\lambda x.zx$ is equivalent to $\lambda y.zy$, since $zy[y \mapsto x] = zx \equiv_\alpha zx$ and $x$ is not a free variable in $zy$.
- $\lambda z.zx$ is not equivalent to $\lambda y.zy$, since $z$ is a free variable in $zy$ and $x$ is not equal to $y$.

### 2.3.4 $\beta$-Reduction

Now, we are ready to discuss $\beta$-reduction. In the $\lambda$-calculus, there is only one way to simplify terms, which is $\beta$-reduction. We use the definition of $\beta$-reduction discussed in [5] and Definition 2.13 defines $\beta$-reduction with $M$ and $N$ being terms in the $\lambda$-calculus.

**Definition 2.13. $\beta$-reduction**

$$(\lambda x.M)N \to M[x \mapsto N] \qquad (\beta)$$

A $\lambda$-term of the form $(\lambda x.M)N$ can be reduced by substituting $x$ by $N$ in $M$. However, without additional rules, we cannot apply $\beta$-reduction to subterms. We use closure rules similar to how José Espírito Santo, Luís Pinto, and Tarmo Uustalu use them in [23]. Closure rules are also discussed in [5] and [22]. For example, the following reduction would not be possible: $\lambda x.(\lambda y.y)x \to \lambda x.x$. Therefore, in Definition 2.14, we define the following closure rules: $\mu$, $\nu$ and $\xi$.

**Definition 2.14. Closure rules for $\beta$-reduction**

$$\frac{M \to M'}{MN \to M'N}\,(\mu) \qquad \frac{N \to N'}{MN \to MN'}\,(\nu) \qquad \frac{M \to M'}{\lambda x.M \to \lambda x.M'}\,(\xi)$$

Now, we define $\rightarrow_\beta$ as $\beta$ closed under $\mu$, $\nu$ and $\xi$. So with $\rightarrow_\beta$ we can use $\beta$-reduction on all subterms if the subterm is of the form $(\lambda x.M)N$. A subterm of the form $(\lambda x.M)N$ is called a $\beta$-redex. In one reduction, we may need to use multiple rules. The $\beta$-redex that is reduced is underlined. For instance, we need to use the $\mu$ and $\xi$ rule in the last example of the following reductions:

**Example 2.15. Usage of Closure Rules**

$$\begin{aligned}
(\underline{\mathbf{II}})(\mathbf{II}) &\rightarrow_\beta \mathbf{I}(\mathbf{II}) && \beta \text{ with } \mu \text{ rule}\\
(\mathbf{II})(\underline{\mathbf{II}}) &\rightarrow_\beta (\mathbf{II})\mathbf{I} && \beta \text{ with } \nu \text{ rule}\\
\lambda x.\underline{\mathbf{I}x} &\rightarrow_\beta \lambda x.x && \beta \text{ with } \xi \text{ rule}\\
(\mathbf{I}(\lambda x.\underline{\mathbf{I}x}))(\mathbf{II}) &\rightarrow_\beta (\mathbf{I}(\lambda x.x))(\mathbf{II}) && \beta \text{ with } \mu \text{ and } \xi \text{ rule}
\end{aligned}$$

We see that $\beta$-reduction is a non-deterministic relation, which means that there can be multiple ways to reduce a term. However, sometimes we would only like to reduce a term in one way and have a deterministic relation. To achieve this, we can define subrelations that do not allow all closure rules.

## 2.4 Call-by-name $\lambda$-calculus

In this section, we discuss the call-by-name $\lambda$-calculus ($\lambda_n$). The reduction rule that is used in this calculus is the same as the $\beta$ reduction rule. However, we name the rule $\beta_n$ (Definition 2.16) to make it clear that it is the reduction rule of $\lambda_n$. While call-by-name avoids evaluating the argument of a function, arguments are re-evaluated every time they are used in the body of the function. Lazy evaluation (call-by-need) stores the evaluation of each argument, so it does not have to re-evaluate the same argument every time it is used. Therefore, an optimised version of call-by-name evaluation is lazy evaluation, which is used in many programming languages. Zena et al. [4] define a call-by-need calculus, but for simplicity, we only use $\lambda_n$ in this thesis.

**Definition 2.16. $\beta_n$-reduction**

$$(\lambda x.M)N \rightarrow M[x \mapsto N] \qquad (\beta_n)$$

The relations we define in this thesis are all similar to the relations discussed in [23] We define the relation $\rightarrow_{\beta_n}$ as $\beta_n$ closed under $\mu$, $\nu$ and $\xi$. The relation $\rightarrow_{\beta_n}$ is the call-by-name reduction relation, which allows reduction of $\lambda$-terms under all term formers. Weak reduction, $\rightarrow_w$, is defined as $\beta_n$ closed under $\mu$ and $\nu$. The only restriction of $\rightarrow_w$ is that it cannot

reduce under $\lambda$'s. Weak reduction will be used later to define a deterministic relation that supports cbn and cbv evaluation in a different $\lambda$-calculus. We also define the relation $\rightarrow_n$ as $\beta_n$ closed under $\mu$. So the relation $\rightarrow_n$ cannot evaluate the argument of a function, since the $\nu$ rule is not allowed with $\rightarrow_n$. Call-by-name evaluation is thus defined as $\rightarrow_n^*$.

Table 2.1 shows the closure rules for the different reduction relations we have defined thus far. In Example 2.17, we give the complete cbn evaluation of $\lambda$-term $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$. The $\beta$-redex that is reduced is underlined in each step. Note that $\rightarrow_n$ is deterministic, so there is always only one way to apply $\rightarrow_n$ on a $\lambda$-term.

| **Relation** | $\boldsymbol{\beta}$ | $\boldsymbol{\mu}$ | $\boldsymbol{\nu}$ | $\boldsymbol{\xi}$ |
|---|---|---|---|---|
| $\rightarrow_{\beta_n}$ | $\beta_n$ | ✓ | ✓ | ✓ |
| $\rightarrow_n$ | $\beta_n$ | ✓ | | |
| $\rightarrow_w$ | $\beta_n$ | ✓ | ✓ | |

Table 2.1: Closure Rules for Different Reduction Relations

**Example 2.17. Cbn evaluation of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$**

$$\begin{aligned}(\underline{\mathbf{I}(\lambda x.\mathbf{I}x)})(\mathbf{II}) &\rightarrow_n \underline{(\lambda x.\mathbf{I}x)(\mathbf{II})}\\ &\rightarrow_n \underline{\mathbf{I}(\mathbf{II})}\\ &\rightarrow_n \underline{\mathbf{II}}\\ &\rightarrow_n \mathbf{I}\end{aligned}$$

## 2.5 Call-by-value $\lambda$-calculus

The call-by-value $\lambda$-calculus ($\lambda_v$) is defined by Gordon Plotkin in [22] and this section defines it similarly. We first define the notion of values. Values are $\lambda$-terms that are fully evaluated. That is, values are $\lambda$-terms that cannot be reduced any further and thus do not contain any function applications. We define values Definition 2.18 as all $\lambda$-terms that are either a variable or an abstraction. We denote values by $V$ or $W$.

**Definition 2.18. Values in $\boldsymbol{\lambda_v}$**

$$V \quad ::= \quad x \mid \lambda x.M$$

The main idea of $\lambda_v$ is that we only reduce a function application if the argument is a value. The $\beta$-reduction rule is defined in Definition 2.19.

**Definition 2.19. $\boldsymbol{\beta_v}$-reduction**

$$(\lambda x.M)V \rightarrow M[x \mapsto V] \qquad (\beta_v)$$

We define a new closure rule, $\nu_<$ [23], which is similar to the $\nu$ rule, but requires the left $\lambda$-term to be a value. In call-by-value evaluation, we first reduce the left $\lambda$-term of a function application, to an abstraction, before reducing the right $\lambda$-term to a value to apply $\beta_v$.

**Definition 2.20.** $\boldsymbol{\nu_<}$

$$\frac{N \to N'}{VN \to VN'}\,(\nu_<)$$

We define call-by-value reduction, $\to_{\beta_v}$, as $\beta_v$ closed under $\mu$, $\nu$ and $\xi$ which allows reduction of $\lambda$-terms under all term formers. The relation $\to_v$ is defined as $\beta_v$, closed under $\mu$ and $\nu_<$. Table 2.2 shows the closure rules for all different reduction relations we have defined thus far.

| **Relation** | $\boldsymbol{\beta}$ | $\boldsymbol{\mu}$ | $\boldsymbol{\nu}$ | $\boldsymbol{\nu_<}$ | $\boldsymbol{\xi}$ |
|---|---|---|---|---|---|
| $\to_{\beta_n}$ | $\beta_n$ | ✓ | ✓ | ✓ | ✓ |
| $\to_n$ | $\beta_n$ | ✓ | | | |
| $\to_w$ | $\beta_n$ | ✓ | ✓ | | |
| $\to_{\beta_v}$ | $\beta_v$ | ✓ | ✓ | ✓ | ✓ |
| $\to_v$ | $\beta_v$ | ✓ | | ✓ | |

Table 2.2: Closure Rules for Different Reduction Relations

Note that $\to_v$ is deterministic, so there is only one way to apply $\to_v$ on a $\lambda$-term. The choice between using $\mu$, $\nu_<$ or no closure rule for $\to_v$ is deterministic. Consider the $\lambda$-term $MN$.

1. If $M$ is an abstraction and $N$ is a value, we can only apply $\beta_v$ directly.
2. If $M$ is an abstraction and $N$ is an application, we can only apply $\nu_<$ to reduce the left $\lambda$-term.
3. If $M$ is an application and $N$ any $\lambda$-term, we can only apply $\mu$ to reduce the left $\lambda$-term.

The closure rule $\nu_<$ forces evaluation from left to right, hence $<$ in the name. First the closure rule $\mu$ is used after which $\nu_<$ is used. In Example 2.21, we give the cbv evaluation of $\lambda$-term $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$. Again, the $\beta$-redex that is reduced is underlined.

**Example 2.21. Cbv evaluation of** $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$

$$\begin{aligned}
\underline{(\mathbf{I}(\lambda x.\mathbf{I}x))}(\mathbf{II}) &\to_v (\lambda x.\mathbf{I}x)(\underline{\mathbf{II}}) \\
&\to_v \underline{(\lambda x.\mathbf{I}x)\mathbf{I}} \\
&\to_v \underline{\mathbf{II}} \\
&\to_v \mathbf{I}
\end{aligned}$$

# Chapter 3

# Call-by-box Lambda Calculus

This chapter introduces the call-by-box lambda calculus ($\lambda_b$), which is a refinement of another $\lambda$-calculus, $\lambda_\Box$ [23]. The calculus $\lambda_\Box$ is a fragment of Bierman and de Paiva's IS4 [7]. The fragment is so restricted that all typing rules are quite simple.

## 3.1 Specification of $\lambda_b$

### 3.1.1 Intuitionistic logic IS4

Intuitionistic logic IS4 is a non-classical logic that is based on the idea of constructive proof. Unlike classical logic, intuitionistic logic does not allow for the law of excluded middle, which states that for every proposition $P$, the statement $P \vee \neg P$ assumed [16]. In intuitionistic logic, a proposition is only considered true if it can be constructively proven. That is, a proposition is true if we can explicitly construct a proof, much like writing a program. This is why programming languages and proofs in systems like Agda and Rocq are based on intuitionistic logic. Since $\lambda$-calculus is the foundation of functional programming, it corresponds with intuitionistic logic via the Curry-Howard correspondence, where programs are equivalent to proofs and types to propositions [18].

The intuitionistic modal logic IS4 is an extension of intuitionistic logic that introduces *modal operator* $\Box$. The $\lambda_\Box$ calculus is a fragment of IS4 that, besides the usual $\lambda$-terms, also uses box (box) and unbox ($\varepsilon$) operators and is strong enough to interpret both call-by-name (cbn) and call-by-value (cbv) lambda calculi [23]. The box is like a little container that delays a computation, making it possible to simulate both cbn and cbv evaluation strategies. In this thesis, we formalize the *boxed lambda calculus* in Agda, denoted as $\lambda_b$, which is as a refinement of the simpler $\lambda_\Box$ calculus.

### 3.1.2 Terms and Types of $\lambda_b$

We now define the types of the $\lambda_b$ calculus as is done in [23]. We indicate the types of $\lambda_b$ with $A$ and $B$, representing types and boxed types respectively. The type $X$ denotes the base type. Note that functions types are of the form $B \to A$ thus the input is always a boxed type.

**Definition 3.1. Types of $\lambda_b$**

$$A \quad ::= \quad X \mid \ B \to A \mid \ B \qquad\qquad B \quad ::= \quad \Box A$$

We give some examples of types in $\lambda_b$. Note that $A$ can be any type that is not boxed, while $B$ can be any type that is boxed. The type $X$ is the base type.

**Example 3.2. Types in $\lambda_b$**

$$X$$
$$\Box(B \to A)$$
$$\Box X \to X$$
$$\Box A \to \Box B$$
$$\Box\Box\Box X \to \Box X$$

Since variables are always boxed, variables in $\lambda_b$ are always inside the unboxing operator $\varepsilon$. Thus, we only have $\varepsilon(x)$ in the grammar with $x$ a variable. Abstractions in $\lambda_b$, take boxed terms as arguments, which is why we define abstractions as $\lambda x.M$, where $x$ is the parameter of a boxed type and $M$ is a term. The rest of the terms are defined in Definition 3.3 and some example terms are given in Example 3.4.

**Definition 3.3. Terms in $\lambda_b$**

$$M, N, P, Q \quad ::= \quad \varepsilon(x) \mid \ \lambda x.M \mid \ MN \mid \ \mathsf{box}(N)$$

**Example 3.4. Terms in $\lambda_b$**

$$\varepsilon(x)$$
$$\lambda x.\varepsilon(x)$$
$$(\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(y))$$
$$(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$$

The typing rules of $\lambda_b$ are defined in Definition 2.3 [23]. The $\Box$-I rule states that if $M$ is of type $A$, then $\mathsf{box}(M)$ is of type $\Box A$. The $\mathrm{Id}_\Box$ rule is the identity rule for boxed variables, which states that if we know that $x$ is a variable of type $\Box A$, we can conclude that $\varepsilon(x)$ is of type $A$. The only way to add variables to the context is by using the $\to$-I rule. These variables

are always boxed, since they are the parameters of abstractions. Due to the grammar of $\lambda_b$, all variables in $\lambda_b$ occur inside the unboxing operator $\varepsilon$ and thus the $\mathrm{Id}_\Box$ rule covers all cases. The $\to$-E rule states that if we have a term $M$ of type $B \to A$ and a term $N$ of type $B$, we can conclude that the application $MN$ has type $A$. It is important that the argument $N$ is a boxed term, which is why we denote its type as $B$.

**Definition 3.5. Typing rules of $\lambda_b$**

$$\frac{}{\Gamma, x : \Box A \vdash \varepsilon(x) : A}\ \mathrm{Id}_\Box$$

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x.M : A \to B}\ \to\text{-I}$$

$$\frac{\Gamma \vdash M : B \to A \qquad \Gamma \vdash N : B}{\Gamma \vdash MN : B}\ \to\text{-E}$$

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \mathsf{box}(M) : \Box A}\ \Box\text{-I}$$

We now demonstrate these typing rules with Example 3.6, where we show that the term $(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$ is well-typed. Note that the argument $\mathsf{box}(\lambda y.\varepsilon(y))$ is boxed. If the argument was not boxed, we would not be able to apply the abstraction $\lambda x.\varepsilon(x)$ to it.

**Example 3.6. Well-typedness of $(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$**

$$\dfrac{\dfrac{\dfrac{}{x : \Box(\Box A \to A) \vdash \varepsilon(x) : \Box A \to A}\ \mathrm{Id}_\Box}{\vdash \lambda x.\varepsilon(x) : \Box(\Box A \to A) \to (\Box A \to A)}\ \to\text{-I} \qquad \dfrac{\dfrac{\dfrac{}{y : \Box A \vdash \varepsilon(y) : A}\ \mathrm{Id}_\Box}{\vdash \lambda y.\varepsilon(y) : \Box A \to A}\ \to\text{-I}}{\vdash \mathsf{box}(\lambda y.\varepsilon(y)) : \Box(\Box A \to A)}\ \Box\text{-I}}{\vdash (\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y)) : \Box A \to A}\ \to\text{-E}$$

## 3.2 Substitution and Free and Bound variables in $\lambda_b$

Since variables always occur inside $\varepsilon$, we need to redefine free and bound variables and substitution. For $\lambda_b$, we have:

**Definition 3.7** (Free and Bound Variables)**.** The free and bound variables of a term $M$ are defined by:

$$\begin{array}{llll} FV_b(\varepsilon(x)) & = \{\varepsilon(x)\} & BV(\varepsilon(x)) & = \emptyset \\ FV_b(\lambda x.M) & = FV(M)\backslash\{\varepsilon(x)\} & BV(\lambda x.M) & = BV(M) \cup \{\varepsilon(x)\} \\ FV_b(MN) & = FV(M) \cup FV(N) & BV(MN) & = BV(M) \cup BV(N) \end{array}$$

**Definition 3.8** (Substitution in $\lambda_b$)**.** The substitution of a term $Q$ for a variable $\varepsilon(x)$ in a term $M$ is defined as follows:

$$\begin{array}{ll} \varepsilon(x)[\varepsilon(x) \mapsto Q] & = Q \\ \varepsilon(y)[\varepsilon(x) \mapsto Q] & = \varepsilon(y) \text{ if } (x \neq y) \\ \mathsf{box}(M)[\varepsilon(x) \mapsto Q] & = \mathsf{box}(M[\varepsilon(x) \mapsto Q]) \\ (MN)[\varepsilon(x) \mapsto Q] & = M[x \mapsto Q]N[x \mapsto Q] \\ (\lambda x.M)[\varepsilon(x) \mapsto Q] & = \lambda x.M \\ (\lambda y.M)[\varepsilon(x) \mapsto Q] & = \lambda z.M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q] \text{ if } (x \neq y) \end{array}$$

with $z$ a variable defined by:

1. If $\varepsilon(x) \notin FV_b(N)$ or $\varepsilon(y) \notin FV_b(M)$ then $z = y$
2. Otherwise, $z$ can be any variable such that $\varepsilon(z) \notin FV(N)$ or $\varepsilon(z) \notin FV(M)$

With this new lambda calculus, it may not be completely clear that substitution preserves well-typedness. We can prove that substitution preserves well-typedness in $\lambda_b$.

**Proposition 3.9** (Preservation of well-typedness)**.**

$$\Gamma, x : \Box A \vdash M : B \text{ and } \Gamma \vdash Q : A \implies \Gamma \vdash M[\varepsilon(x) \mapsto Q] : B$$

**Proof 3.10.** We prove by structural induction on the term $M$ that this substitution preserves well-typedness.

1. $$\varepsilon(x)[\varepsilon(x) \mapsto Q] = Q$$

   Since $Q$ is well-typed, and $Q$ is the result of the substitution, we have that well-typedness is preserved.

2. $$\varepsilon(y)[\varepsilon(x) \mapsto Q] = \varepsilon(y) \quad \text{if } x \neq y$$

   We have that $x \neq y$, and the result of the substitution is $\varepsilon(y)$, which is well-typed.

3.
$$\mathsf{box}(M)[\varepsilon(x) \mapsto Q] = \mathsf{box}(M[\varepsilon(x) \mapsto Q])$$

We have that $\mathsf{box}(M)$ is well-typed and thus $M$ is well-typed because of the $\Box$-I rule. By the inductive hypothesis, we have that $M[\varepsilon(x) \mapsto Q]$ is well-typed. We also have that $\mathsf{box}(M[\varepsilon(x) \mapsto Q])$ is well-typed because of the $\Box$-I rule. Thus, we have that $\mathsf{box}(M)[\varepsilon(x) \mapsto Q]$ is well-typed.

4.
$$(MN)[\varepsilon(x) \mapsto Q] = (M[\varepsilon(x) \mapsto Q])\ (N[\varepsilon(x) \mapsto Q])$$

We have that $MN$ is well-typed and thus $M$ and $N$ are well-typed. By the inductive hypothesis, we get that $M[\varepsilon(x) \mapsto Q]$ is well-typed, and $N[\varepsilon(x) \mapsto Q]$ is well-typed. We also have that application of two well-typed terms is well-typed, so we have that $(MN)[\varepsilon(x) \mapsto Q]$ is well-typed.

5.
$$(\lambda x.M)[\varepsilon(x) \mapsto Q] = \lambda x.M$$

We have that $\lambda x.M$ is well-typed and thus $M$ is well-typed. Since $\varepsilon(x)$ is a bound variable in $\lambda x.M$, no substitution occurs and the result is the well-typed term $\lambda x.M$.

6.
$$(\lambda y.M)[\varepsilon(x) \mapsto Q] = \lambda z.(M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q]) \quad \text{if } x \neq y$$

We have that $\lambda y.M$ is well-typed and thus $M$ is well-typed. By the inductive hypothesis, we have that $M[\varepsilon(y) \mapsto \varepsilon(z)]$ and $M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q]$ are well-typed. We also have that abstraction of a well-typed term is well-typed, so we have that
$\lambda z.(M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q])$ is well-typed. $\Box$

With this definition of substitution, we define the reduction rule for $\lambda_b$ in Definition 3.11. In $\lambda_b$, the arguments of abstractions are always boxed, so we can only apply abstractions to boxed terms, which makes sense since the parameters like $x$ are always boxed. Since variables only occur insides $\varepsilon$, we can replace those, including $\varepsilon$, with the corresponding unboxed term.

**Definition 3.11** ($\beta_b$)**.** In $\lambda_b$, $\beta$ reduction is defined as follows:

$$(\lambda x.M)\mathsf{box}(N) \to M[\varepsilon(x) \mapsto N] \qquad (\beta_b)$$

For completeness, we need to define a new closure rule. This new rule only applies to the $\lambda_b$, and allows reduction under boxed terms. We call this rule $\zeta$, and Definition 3.12 defines it.

**Definition 3.12.** $\zeta$

$$\frac{M \to_{\beta_b} N}{\mathsf{box}(M) \to_{\beta_b} \mathsf{box}(N)}\,(\zeta)$$

The relations defined here are from [23] and form the reduction and evaluation relations of $\lambda_b$. The relation $\to_{\beta_b}$ allows reduction under all term formers, including boxes. We also define three other relations: $\beta_b$ closed under $\mu$ and $\nu$, $\beta_b$ closed under $\mu$ and $\beta_b$ closed under $\nu$. We call them $\to_{we}$, $\to_{b_n}$ and $\to_{b_v}$ respectively. Table 3.1 is an overview of all relations. The relation $\to_{we}$ is weak since values do not reduce. That is, abstractions and terms of the form $\varepsilon(x)$ do not reduce. The relation $\to_{we}$ is also external as boxes do not reduce. This relation is called call-by-box (cbb) evaluation. We will see after defining the embeddings that $\to_{we}$ collapses to $\to_{b_n}$ in Girard's embedding and that $\to_{we}$ collapses to $\to_{b_v}$ in Gödel's embedding.

| **Relation** | $\boldsymbol{\beta}$ | $\boldsymbol{\mu}$ | $\boldsymbol{\nu}$ | $\boldsymbol{\nu_<}$ | $\boldsymbol{\xi}$ | $\boldsymbol{\zeta}$ |
|---|---|---|---|---|---|---|
| $\to_{\beta_n}$ | $\beta_n$ | ✓ | ✓ | ✓ | ✓ | |
| $\to_n$ | $\beta_n$ | ✓ | | | | |
| $\to_w$ | $\beta_n$ | ✓ | ✓ | | | |
| $\to_{\beta_v}$ | $\beta_v$ | ✓ | ✓ | ✓ | ✓ | |
| $\to_v$ | $\beta_v$ | ✓ | | ✓ | | |
| $\to_{\beta_b}$ | $\beta_b$ | ✓ | ✓ | ✓ | ✓ | ✓ |
| $\to_{we}$ | $\beta_b$ | ✓ | ✓ | ✓ | | |
| $\to_{b_n}$ | $\beta_b$ | ✓ | | | | |
| $\to_{b_v}$ | $\beta_b$ | | ✓ | | | |

Table 3.1: Closure Rules for Different Reduction Relations

# Chapter 4

# Embeddings in $\lambda_b$

This chapter discusses the modal embeddings of the call-by-name $\lambda$-calculus ($\lambda_n$), and the call-by-value $\lambda$-calculus ($\lambda_v$) into the shared target, call-by-box $\lambda$-calculus ($\lambda_b$). We also give propositions about the embeddings, which will be proven in Agda later. These propositions state that reduction and evaluation are preserved by the embeddings, and have been proven several times in different contexts, for example, using linear logic or thunks [13, 19, 15]. However, $\lambda_b$ is a refined version of $\lambda_\Box$ [23]. We formalise these embeddings and prove their propositions in Agda.

## 4.1 Girard's embedding

In this section, we present Girard's embedding, which is a method to translate intuitionistic logic into linear logic [14]. Jean-Yves Girard introduced this embedding to represent intuitionistic logic in a linear framework, where resources are managed explicitly. Linear logic can be seen as a refinement of classical logic, where assumptions - or resources - are managed explicitly [25]. In particular, linear logic distinguishes between assumptions that should be used exactly once and those that can be used any number of times [25], including zero times. However, this thesis does not focus on linear logic, but on the way Girard's embedding can be used to represent the call-by-name $\lambda$-calculus in the modal call-by-box $\lambda$-calculus, $\lambda_b$.

In general, Girard's embedding is not limited to linear logic. It is more general technique to represent one logical system in a different framework [26]. The embedding can be used to represent one logical system in another framework that has a way to manage resources or assumptions. In the context of $\lambda_n$, parameters of functions can be seen as resources that are consumed when the function is applied [19]. These resources can be used as many times as needed, hence the connection with Girard's embedding and linear logic.

The model embedding of the $\lambda_n$ into the $\lambda_b$ is given by Girard's transla-

tion, denoted as $(\cdot)^\circ$. Thus, $M^\circ$ is the translation of a term $M$ from the $\lambda_n$ to the $\lambda_b$, and $A^\circ$ is the translation of a type $A$ from the $\lambda_n$ to the $\lambda_b$. The translation is inductively defined in Figure 4.1 for both terms and types. We also give the translation of $\lambda$-term $\mathbf{I}x$ in Example 4.1.

$$\begin{aligned} X^\circ &= X & x^\circ &= \varepsilon(x) \\ (A_1 \to A_2)^\circ &= \Box {A_1}^\circ \to {A_2}^\circ & (\lambda x.M)^\circ &= \lambda x.M^\circ \\ & & (MN)^\circ &= M^\circ \mathsf{box}(N^\circ) \end{aligned}$$

Figure 4.1: Girard's translation from $\lambda_n$ to $\lambda_b$.

**Example 4.1. $(\mathbf{I}x)^\circ$**

$$\begin{aligned} (\mathbf{I}x)^\circ &= (\lambda x.x)^\circ \mathsf{box}(x^\circ) \\ &= (\lambda x.x^\circ)\mathsf{box}(x^\circ) \\ &= (\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(x)). \end{aligned}$$

From Girard's translation, we see that a boxed $\lambda$-term can never occur on its own. Boxed terms are always part of an application, where the left-hand side is a term in $\lambda_b$ and the right-hand side is the boxed term. Therefore, the image of Girard's translation is a subset of the $\lambda_b$ given by the following grammar:

$$M, N \quad ::= \quad \varepsilon(x) \mid \ \lambda x.M \mid \ M\mathsf{box}(N)$$

We can reason that the relation $\to_{we}$ collapses to $\to_{b_n}$ when we consider the image of Girard's translation. Since application in Girard's image is always of the form $M\mathsf{box}(N)$, the right-hand side of the application can never be reduced without the use of the $\zeta$ rule. In $\to_{we}$, the only closure rules allowed are $\mu$ and $\nu$. It is clear that the $\nu$ rule can never be applied in Girard's image as it is not possible to reduce under boxed terms. Thus, the only closure rule that can be applied is the $\mu$ rule, and $\to_{we}$ collapses to $\to_{b_n}$ in Girard's image.

We give a small example to clarify everything. Consider the $\lambda$-term $\mathbf{I}x$. It holds that $\mathbf{I}x \to_n x$ in $\lambda_n$. Example 4.2 shows that $(\mathbf{I}x)^\circ \to_{b_n} x^\circ$ also holds.

**Example 4.2. $(\mathbf{I}x)^\circ$ under $\rightarrow_{b_n}$**

$$\begin{array}{rcl}(\mathbf{I}x)^\circ & = & (\lambda x.x)^\circ \mathsf{box}(x^\circ) \\ & = & (\lambda x.x^\circ)\mathsf{box}(x^\circ) \\ & = & (\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(x)) \\ & \rightarrow_{b_n} & (\lambda x.\varepsilon(x))[\varepsilon(x) \mapsto \varepsilon(x)] \\ & = & \varepsilon(x) \\ & = & x^\circ\end{array}$$

We now discuss the properties of Girard's translation. Girard's translation preserves the reduction and evaluation relations of $\lambda_n$. Preservation of reduction is the first proposition and is defined by proposition 4.4. More specifically, it states that if a $\lambda$-term $M$ reduces to $N$ in $\lambda_n$, then $M^\circ$ reduces to $N^\circ$ in $\lambda_b$. The proof of this proposition is rather simple and is given in the chapter about the Agda formalisation, section 5.3.3, though it requires a helper lemma, lemma 4.3, which states that Girard's translation preserves and reflects substitution. The proof of this helper lemma is rather complex, because of the definition of substitution.

**Lemma 4.3. Preservation of substitution** under Girard's translation.

For all $\lambda$-terms $M$, $N$ and $x$, $M^\circ[N^\circ \mapsto \varepsilon(x)] \equiv (M[N \mapsto x])^\circ$.

**Proposition 4.4. Preservation of reduction** under Girard's translation.

For all $\lambda$-term $M$ and $N$, if $M \rightarrow_{\beta_n} N$ in $\lambda_n$, then $M^\circ \rightarrow_{\beta_b} N^\circ$.

The second proposition about Girard's translation is proposition 4.5. It states that if a $\lambda$-term $M$ evaluates to $\lambda$-term $N$ with $\rightarrow_n$ in $\lambda_n$ in one step, then the translation of $M$ under Girard's embedding also evaluates to the translation of $N$ under Girard's embedding with $\rightarrow_{b_n}$ in one step. From this proposition, it easily follows that if $M$ evaluates to $N$ by call-by-name in $\lambda_n$, then $M^\circ$ evaluates by call-by-box to $N^\circ$ in $\lambda_b$.

**Proposition 4.5. Preservation of evaluation** under Girard's translation.

For all $\lambda$-term $M$ and $N$, if $M \rightarrow_n N$ in $\lambda_n$, then $M^\circ \rightarrow_{b_n} N^\circ$.

## 4.2 Gödel's Embedding

In this section, we present Gödel's embedding, introduced by Kurt Gödel. Gödel introduced this embedding as a method to translate intuitionistic logic into modal logic $\mathsf{S4}$ [24]. More specifically, an embedding to represent intuitionistic logic in a modal framework, where box modality, $\Box$ is used to indicate provability. That is, $\Box P$ can be read as $P$ is provable [24].

In general, Gödel's embedding is used to represent one more restricted logical system in a stronger framework [20]. This thesis uses Gödel's embedding to represent the call-by-name $\lambda$-calculus in the modal call-by-box $\lambda$-calculus, $\lambda_b$ [23]. The intuition behind Gödel's embedding is that boxes are used to represent values, which are already evaluated expressions [23].

In the model embedding of $\lambda_v$ into $\lambda_b$, we make use of an abbreviation, which we call $\mathsf{raise}$ similar to [23]. The idea of $\mathsf{raise}$ is to swap two lambda terms, while removing a box of one $\lambda$-term. Its use will become clear later in this section when we give some examples. We define $\mathsf{raise}$ as follows:

$$\mathsf{raise}(M) := \lambda z.\varepsilon(z)M$$

The model embedding of $\lambda_v$ into $\lambda_b$ is given by Gödel's translation, denoted as $(\cdot)^*$. Thus, $M^*$ is the translation of a term $M$ from $\lambda_v$ to $\lambda_b$, and $A^*$ is the translation of a type $A$ from the $\lambda_v$ to the $\lambda_b$. We also define $(\cdot)^\bullet$ for the translation of values and types in $\lambda_v$. The embedding $(\cdot)^*$ and $(\cdot)^\bullet$ are mutually inductively defined in Figure 4.2 for both terms and types. That is, we define $(\cdot)^*$ in terms of $(\cdot)^\bullet$ and vice versa. We also give the translation of $\lambda$-term $\mathbf{I}x$ in Example 4.6.

$$A^* = \Box A^\bullet \qquad\qquad V^* = \mathsf{box}(V^\bullet)$$
$$(MN)^* = \mathsf{raise}(N^*)M^*$$

$$X^\bullet = X \qquad\qquad x^\bullet = \varepsilon(x)$$
$$(A_1 \to A_2)^\bullet = \Box A_1{}^\bullet \to \Box A_2{}^\bullet \qquad (\lambda x.M)^\bullet = \lambda x.M^*$$

Figure 4.2: Gödel's translation from $\lambda_v$ to $\lambda_b$.

**Example 4.6. $(\mathbf{I}x)^*$**

$$\begin{aligned}(\mathbf{I}x)^* &= \mathsf{raise}(x^*)(\lambda x.x)^* \\ &= \mathsf{raise}(\mathsf{box}(x^\bullet))\mathsf{box}((\lambda x.x)^\bullet) \\ &= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.x^*) \\ &= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(x^\bullet)) \\ &= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x)))\end{aligned}$$

Let us now reason about the image of Gödel's translation. From the translation, it follows that all applications in the image of Gödel's translation are of the form $\mathsf{raise}(M')M$, where both $M'$ and $M$ are terms in $\lambda_b$. Every term in the image that is not an application, is of the form $\mathsf{box}(V)$, with $V$ a value in $\lambda_b$. Thus, the image of Gödel's translation is a subset of $\lambda_b$ given by the following grammar:

$$M \quad ::= \quad \mathsf{box}(V) \mid \ \mathsf{raise}(M')M \qquad\qquad V \quad ::= \quad \varepsilon(x) \mid \ \lambda x.M$$

Now, we reason that the relation $\rightarrow_{we}$ collapses to $\rightarrow_{b_v}$ when we consider the image of Gödel's translation. We do this by showing that the only closure rule that can be applied on each type of term in Gödel's image is the $\nu$ rule.

1. If a term in Gödel's image is of the form $\mathsf{box}(V)$, we cannot apply any reduction rule, since we do not allow reductions under boxed terms. Besides, $V$ is a value, which cannot be reduced.

2. If a term in Gödel's image is of the form $\mathsf{raise}(M')M$, we can only apply the $\nu$ rule. We have that $\mathsf{raise}(M') = \lambda z.\varepsilon(z)M'$, which is a value. Thus, $\mathsf{raise}(M')$ cannot be reduced and the $\mu$ rule cannot be applied. The only closure rule that can be applied is the $\nu$ rule.

We give a small example to clarify everything. Consider the $\lambda$-term $\mathbf{I}x$. It holds that $\mathbf{I}x \rightarrow_v x$ in $\lambda_n$. Example 4.2 shows that $(\mathbf{I}x)^* \rightarrow_{b_v}^* x^*$ also holds.

**Example 4.7. $(\mathbf{I}x)^*$**

$$\begin{aligned}
(\mathbf{I}x)^* &= \mathsf{raise}(x^*)(\lambda x.x)^* \\
&= \mathsf{raise}(\mathsf{box}(x^\bullet))\mathsf{box}((\lambda x.x)^\bullet) \\
&= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.x^*) \\
&= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(x^\bullet)) \\
&= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x))) \\
&= (\lambda z.\varepsilon(z)\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x))) \\
&\rightarrow_{b_v} (\lambda x.\mathsf{box}(\varepsilon(x)))\mathsf{box}(\varepsilon(x)) \\
&\rightarrow_{b_v} \mathsf{box}(\varepsilon(x)) \\
&= \mathsf{box}(x^\bullet) \\
&= x^*
\end{aligned}$$

We now discuss the properties of Gödel's translation. Just like Girard's translation, Gödel's translation preserves reduction and evaluation. To proof these properties, we again need a helper lemma that is similar to the one used for Girard's translation. The helper lemma, lemma 4.8, states that Gödel's translation preserves and reflects substitution. The lemma is divided into two parts - one concerning substitution within values, and the other dealing with substitution in any $\lambda$-term.

**Lemma 4.8. Preservation and Reflection of substitution** under Gödel's translation. For all $\lambda$-terms $M$, values $V$, $W$, and variables $x$, it holds that

$$M^*[V^\bullet \mapsto \varepsilon(x)] \equiv (M[V \mapsto x])^* \textit{ and } M^\bullet[W^\bullet \mapsto \varepsilon(x)] \equiv (W[V \mapsto x])^\bullet$$

Preservation of reduction of Gödel's translation is slightly more complicated than Girard's translation. This is because Gödel's translation makes use of the raise abbreviation, which swaps two terms. However, this swapping takes an extra step while reducing a $\lambda$-term. Thus, proposition 4.9 states that if a $\lambda$-term $M$ in $\lambda_v$ reduces to $N$ in one step, then $M^*$ reduces to $N^*$ in $\lambda_b$ in two steps.

**Proposition 4.9. Preservation of reduction** under Gödel's translation.

For all $\lambda$-terms $M$ and $N$, if $M \to_{\beta_n} N$ in $\lambda_v$, then $M^* \to^2_{\beta_b} N^*$.

The two steps that are taken next to each other are always similar and can be merged together into one step. Let us reason about this.

Let us say that $\lambda$-term $P$ in $\lambda_v$ reduces to $\lambda$-term $Q$. We have that $P$ is of the form $(\lambda x M)V$ and thus $Q$ is of the form $M[x \mapsto V]$. In Gödel's image, we have the following:

$$P^* \equiv \mathsf{raise}(V^*)(\lambda x.M)^* \equiv \mathsf{raise}(\mathsf{box}(V^\bullet))\mathsf{box}(\lambda x M^*)$$

$$\begin{aligned}\mathsf{raise}(\mathsf{box}(V^\bullet))\mathsf{box}(\lambda x M^*) &\equiv (\lambda z.\varepsilon(z)\mathsf{box}(V^\bullet))\mathsf{box}(\lambda x M^*)\\ &\to_{\beta_b} (\lambda x M^*)\mathsf{box}(V^\bullet)\\ &\to_{\beta_b} M^*[\varepsilon(x) \mapsto V^\bullet]\end{aligned}$$

So, we have that $\mathsf{raise}(\mathsf{box}(V^\bullet))\mathsf{box}(\lambda x M^*) \to^2_{\beta_b} M^*[\varepsilon(x) \mapsto V^\bullet]$. More generally, Definition 4.10 defines the relation $\to_{\beta_{b2}}$.

**Definition 4.10. Gödel's two-step reduction relation $\to_{\beta_{b2}}$.**

$$\mathsf{raise}(\mathsf{box}(N))\mathsf{box}(\lambda x M) \to_{\beta_{b2}} M[\varepsilon(x) \mapsto N] \qquad (\to_{\beta_{b2}})$$

With this new reduction relation, we can also formulate the preservation of reduction in one step. This is done in proposition 4.11.

**Proposition 4.11. Preservation of reduction** under Gödel's translation with $\to_{\beta_{b2}}$ reduction rule.

For all $\lambda$-terms $M$ and $N$, if $M \to_{\beta_v} N$ in $\lambda_v$, then $M^* \to_{\beta_{b2}} N^*$.

Preservation of evaluation is more difficult to prove than preservation of reduction. Proposition 4.5 states that if a $\lambda$-term $M$ evaluates to a value $V$ in $\lambda_v$ in any number of $\to_v$ steps, then $M^*$ evaluates to $V^*$ in $\lambda_b$ using any number of $\to_{b_v}$ steps. Alternatively, we can also say that if a $\lambda$-term evaluates by call-by-value to a value, then the translation of that $\lambda$-term under Gödel's embedding evaluates by call-by-box to the translation of that value under Gödel's embedding.

Due to time limitations, we did not prove proposition 4.12 in the Agda formalization. However, a less formal proof of this proposition is given by José Espírito Santo, Luís Pinto, and Tarmo Uustalu in [23].

**Proposition 4.12. Preservation of evaluation** under Gödel's translation.

For all $\lambda$-terms $M$ and values $V$, if $M \to_v V$ in $\lambda_v$, then $M^* \to_{b_v} V^*$.

# Chapter 5

# Formalisation in Agda

## 5.1 Intro to Agda

This section, aims to provide a brief introduction to Agda, a dependently typed functional programming language and proof assistant. Agda uses strong typing and dependent types, and thus it can be used to write proofs constructively, which can be run as programs [21]. In order to understand the formalisation of $\lambda_b$ in Agda, we discuss the following basic concepts: data types, parameterised data types, functions, dependent data types, implicit arguments, and hole-based programming [2].

### 5.1.1 Data Types and Functions

Data types, can be seen as a collection of values that have something in common. This collection is usually structured in some kind of way, such as a list or a tree [21]. Elements of a data type can be constructed using constructors, which are functions that return a value of the data type. In Agda, we can create a data type with the data keyword, followed by the name of the data type and its constructors [2].

Example 5.1 shows an example of a simple inductive data type: the natural numbers. It defines two constructors:

- zero, representing the base case (the number zero),
- suc, which takes a natural number and returns its successor.

The keyword Set in the type signature indicates the type of type. This is analogous to * in Haskell, which denotes the kind of types.

**Example 5.1. Natural numbers as a data type in Agda:**

```
data ℕ : Set where
  zero : ℕ
  suc : ℕ → ℕ
```

This example shows how the natural numbers can be defined in Agda by using an inductive data type with the constructors zero and suc.

Data types in Agda can also have parameters. This makes them more flexible and allows for the creation of more complex data types. Example 5.2 shows such a data type, which is a list that can hold elements of any type $A$. We can make a list of natural numbers, which will have the type List $\mathbb{N}$, or a list of Booleans, which will have the type List Bool. Note that the parameter $A$ occurs before the colon, indicating that it is a type parameter.

**Example 5.2. List as a parameterised data type in Agda:**:

```
data List (A : Set) : Set where
  [] : List A
  _::_ : A → List A → List A
```

This example shows how a list can be defined in Agda by using a parameterised data type with the constructors [] and _::_.

Functions in Agda are defined using pattern matching, similar to Haskell. Each function has a type signature that defines the type of the function and its arguments. After the type signature, a number of equations follows, defining the function for each pattern, where functions can be recursive. Example 5.3 shows such a function, which doubles a natural number.

Functions can also be higher order, taking other functions as arguments or returning functions as results. Function application of function $F$ to argument $x$ is written as $F\ x$ in Agda. Infix functions are denoted with underscores, such as _ :: _ in Example 5.2, where it is the constructor for the list data type. The first argument can be put on the left side of the constructor and the second argument on the right side, such as in $x :: xs$.

**Example 5.3. Function that doubles a natural number in Agda:**

```
double : ℕ → ℕ
double zero = zero
double (suc n) = suc (suc (double n))
```

This example shows how a function can be defined in Agda by using pattern matching on the natural numbers.

Example 5.4 presents a dependent data type, which is a vector that has a length that is encoded in its type. In this example, we see that the type of a vector depends on a natural number, which is different for both constructors. The dependent type occurs after the colon, which is $\mathbb{N}$ in this case. Dependent types differ from parameterised types in that they can vary between constructors, whereas parameterised types use a single type parameter shared by all constructors.

The constructor _ :: _ in Example 5.4 is also an example of a dependent function. That is, the return type of _ :: _ depends on the value of the implicit argument $n$. The argument $n$ is implicit, denoted by the curly braces around it.

We see that [] constructs a vector of length zero, resulting in type Vector $A$ zero, while the constructor _ :: _ constructs a vector of length $n+1$, resulting in type Vector $A$ (suc $n$). Note that the implicit argument $n$ can be inferred from the input type Vector $A$ $n$.

**Example 5.4. Vector as a dependent data type in Agda:**

```
data Vector (A : Set) : ℕ → Set where
  [] : Vector A zero
  _::_ : {n : ℕ} → A → Vector A n → Vector A (suc n)
```

This example shows how a vector can be defined in Agda by using a dependent data type with the constructors [] and _::_.

### 5.1.2 Hole-Based Programming

Agda is made to be used with the Emacs text editor, and the combination of Agda and Emacs provides a way to construct definitions and proofs interactively [21, 2, 1], which is called hole-based programming. Terms can be constructed step by step, using question marks as placeholders. These turn into holes, indicating incomplete definitions. It is best to show this with an example. Note that we could use more holes in Example 5.5, but we feel like this example is sufficient for understanding the concept of hole-based programming.

**Example 5.5. Hole-based programming in Agda:**
This example defines the function plus that adds two natural numbers. It defines the function with a question mark, which we can fill in later. The type signature is already given.

```
plus : ℕ → ℕ → ℕ
plus = ?
```

Pressing the keys `C-c C-l` (while pressing the control key, typing c and then typing l) loads the file and transforms the question mark into a hole.

```
plus : ℕ → ℕ → ℕ
plus = {!   !}
```

We can now let Agda give the arguments of the plus function by pressing `C-c C-c`. The following prompt is shown:

```
pattern variables to case (empty for split on result):
```

Since we do not have any arguments yet, we can press enter.

$$\begin{aligned}&\mathsf{plus} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}\\&\mathsf{plus}\ x\ x_1 = \{!\quad !\}\end{aligned}$$

Now, we would like to do a case distinction on the first argument. We let Agda do this by pressing `C-c C-c` again. The same prompt will be shown as before, but now we type the first argument $x$.

$$\begin{aligned}&\mathsf{plus} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}\\&\mathsf{plus}\ \mathsf{zero}\ x_1 = \{!\quad !\}\\&\mathsf{plus}\ (\mathsf{suc}\ x)\ x_1 = \{!\quad !\}\end{aligned}$$

Inside the first hole, we press `C-c C-,`. This gives us information about the type of the hole and all the variables in scope, which are the variables that we can use to fill in the hole. We get something like the following output:

```
Goal: ℕ
------------------------
x₁ : ℕ
```

This tells us we need to produce a natural number and that we can use the variable $x_1$ inside this hole. Since we have that $0 + x_1 = x_1$, we fill in the hole with $x_1$, which can be done by typing $x_1$ inside the hole and pressing `C-c C-space`. The hole will be filled in and Agda will check if the type of the hole is correct. Pressing `C-c C-f` will move the cursor forward to the next hole.

$$\begin{aligned}&\mathsf{plus} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}\\&\mathsf{plus}\ \mathsf{zero}\ x_1 = x_1\\&\mathsf{plus}\ (\mathsf{suc}\ x)\ x_1 = \{!\quad !\}\end{aligned}$$

Typing `C-c C-,` in this second hole gives us the type of the next hole. The output will be similar to the following:

```
Goal: ℕ
------------------------
x₁ : ℕ
x : ℕ
```

The goal is to produce a natural number, which is the result of an addition. Since one of the arguments is suc $x$, we know the result is not zero. The result should a natural number that uses the suc constructor. We fill in the hole with suc ? and type `C-c C-space`. We closed a hole and made a new hole, which is the argument of the suc constructor. We now have the following code:

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = x₁
plus (suc x) x₁ = suc {!   !}
```

We can fill in the last hole with plus $x$ $x_1$ and press `C-c C-space` to close the hole. The final code looks like this:

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = x₁
plus (suc x) x₁ = suc (plus x x₁)
```

Example 5.5 shows how to use the commands `C-c C-l`, `C-c C-c`, `C-c C-f`, `C-c C-space`, and `C-c C-,` to interactively develop a function in Agda. However, there many more commands available in Agda to help with hole-based programming. For a complete list of commands, we refer to the Agda documentation about Emacs Mode [1].

## 5.2 Challenges of the Formalisation

This sections discusses the challenges of the formalisation of $\lambda_b$ in Agda. The main challenge is dealing with variables and substitution. For variables, we used de Bruijn indices, which are a way to represent variables without names. The definition of substitution is based on the definition of substitution in the PLFA book [27], and is based on the definition by Conor McBride [6]. The formalisation of $\lambda_b$ in Agda is restricted to well-typed terms, which avoids handling ill-typed terms. Another challenge is explicit weakening and renaming, which is needed to handle the contexts of $\lambda$-terms correctly. Two $\lambda$-terms with slightly different contexts are not the same, and we need to take this into consideration in the formalisation.

### 5.2.1 De Bruijn Indices

De Bruijn indices, introduced by Nicolaas Govert de Bruijn, are a method to eliminate the need for variable names in the $\lambda$-calculus [12]. This is useful because now all $\lambda$-terms that are $\alpha$-equivalent look the same. We do not have to worry about variables with the same name that refer to different variables, or about variable capture, simplifying the formalisation

in Agda. The definition of substitution becomes simpler because of this. This subsection first explains the concept of de Bruijn indices, and then shows how they are implemented in the formalisation.

The $\lambda$-terms that use de Bruijn indices do not use variable names, but integers. We precede the integers with # to prevent confusion if the index is higher than 9. Without #, 10 could be confused with two indices 1 and 0 that occur next to each other. These integers represent the "distance" to the corresponding $\lambda$. Example 5.6 gives representations of $\lambda$-terms without free variables, using de Bruijn indices. These terms are said to be closed.

**Example 5.6.** Closed $\lambda$-terms represented with **de Bruijn indices**:

$\lambda x.x \equiv \lambda.\#0$
$\lambda x.\lambda y.x \equiv \lambda.\lambda.\#1$
$\lambda x.\lambda y.y \equiv \lambda.\lambda.\#0$
$\lambda x.x\lambda y.y \equiv \lambda.\#0\lambda.\#0$

Representing open $\lambda$-terms ($\lambda$-terms with free variables) using de Bruijn indices is a little more complex. First, we represent the $\lambda$-term with de Bruijn indices, ignoring the free variables. Then, we give each free variable a unique index, that is larger than the largest index of the bound variables. Example 5.7 demonstrates this.

**Example 5.7.** Open $\lambda$-terms represented with **de Bruijn indices**:

$x\lambda y.y \equiv x\lambda.0 \equiv \#1\lambda.\#0$
$x\lambda y.x \equiv x\lambda.x \equiv \#1\lambda.\#1$
$\lambda x.\lambda t.xt(\lambda s.w)wy \equiv \lambda.\lambda.\#1\#0(\lambda.w)wy \equiv \lambda.\lambda.\#1\#0(\lambda.\#4)\#3\#2$

The last $\lambda$-term in this example is a little more complex and is the example that de Bruijn gave in his paper [12]

In the formalisation, we will represent de Bruijn indices similar to how natural numbers are represented in Agda. De Bruijn index 0 is represented as `Z`, de Bruijn index 1 as `S Z`, de Bruijn index 2 as `S (S Z)`, and so on.

### 5.2.2 Well-typed Terms

This subsection discusses the restriction to well-typed terms in the formalisation of $\lambda_b$. The restriction to well-typed terms is important because it avoids all complications that arise from ill typed terms. In particular, in the context of evaluation, it makes sense to restrict to well-typed terms, because ill-typed terms do not have a normal form. Evaluation is only defined for well-typed terms, and thus the restriction to well-typed terms makes sense.

The formalisation uses an intrinsically typed approach where terms are defined together with their types. So $\lambda$-terms in the formalisation are automatically associated with a type, and we can only construct well-typed terms. This approach is also referred to as Church style typing, and was introduced by Alonzo Church in [9].

The formalisation is based on the representation proposed by Thorsten AltenKirch and Bernhard Reus in [3]. We briefly go over the types and $\lambda$-terms in the formalisation, and how they are defined. Definition 5.8 gives the formal definition of types in the $\lambda$-calculus.

**Definition 5.8. Types** in the $\lambda$-calculus:

```
data Type : Set where
  _⇒_ : Type → Type → Type
  X : Type
```

Terms in the $\lambda$-calculus are defined together with a context, which can be seen as a list of Types. We do not need to name the variables in the context, because we use de Bruijn indices to refer to them. So, the context is a list of types, where each type corresponds to a variable in the context. Definition 5.9 gives the formal definition of contexts in the $\lambda$-calculus. This construction looks somewhat similar to the list data type that we gave as example in Example 5.2.

**Definition 5.9. Context** in the $\lambda$-calculus:

```
data Context : Set where
  ∅ : Context
  _,_ : Context → Type → Context
```

We now define de Bruijn indices that refer to the variables in the context. They can be seen as a proof that some type is in the context. If $A$ occurs in the context, then $A \in \Gamma$ denotes that $A$ is in the context $\Gamma$, and de Bruijn index is the proof of this. It gives the location in the context where the type occurs. The de Bruijn index Z refers the most recently introduced variable in the context, which appears on the right most position of the context. The de Bruijn index S Z refers to the second most recently introduced variable, and so on.

**Definition 5.10. De Bruijn indices** in the $\lambda$-calculus:

```
data _∈_ : Type → Context → Set where
  Z : ∀ {Γ A} → A ∈ (Γ , A)
  S : ∀ {Γ A B} → A ∈ Γ → A ∈ (Γ , B)
```

Definition 5.11 gives the formal definition of terms in the $\lambda$-calculus. The $\lambda$-terms are defined by the context they are in and the type they have. For example, $\Gamma \vdash A$ denotes the $\lambda$-terms that have type $A$ in the context $\Gamma$.

With the constructors ', $\lambda$ and $\cdot$, we can construct $\lambda$-terms. We cannot use the $\lambda$ for $\lambda$ because it is already used in Agda for making functions. The ' constructor is used to construct $\lambda$-terms that are variables if we can provide evidence that the variable is in the context. Abstractions are constructed with the $\lambda$ constructor, which takes a context with more than one variable, and creates a new $\lambda$-term that has one less variable in the context. This new $\lambda$-term is the abstraction of the original $\lambda$-term. The $\cdot$ constructor is used to construct applications of $\lambda$-terms. Given a $\lambda$-term that has a function type and an argument of the input type of the function type, $\cdot$ constructs a new $\lambda$-term of the return type of the function type.

Examples of $\lambda$-terms are given in Example 5.12. Note that we need to give the type of the $\lambda$-term, which is of the form $\Gamma \vdash A$, and that we provide evidence for this type when constructing the $\lambda$-term.

**Definition 5.11. Data type of $\lambda$-terms in Agda:**

```
data _⊢_ : Context → Type → Set where
  ‘ : ∀ {Γ A}
    → A ∈ Γ
      -------
    → Γ ⊢ A

  ƛ : ∀ {Γ A B}
    → Γ , A ⊢ B
      --------------
    → Γ ⊢ A ⇒ B

  _·_ : ∀ {Γ A B}
    → Γ ⊢ A ⇒ B
    → Γ ⊢ A
      --------------
    → Γ ⊢ B
```

**Example 5.12. $\lambda$-terms in Agda:** In this example, we show how to construct $\lambda$-terms in Agda. We also show in comments to what $\lambda$-terms they correspond if they are written in the traditional style.

```
t₁ : ∅ , X ⊢ X
t₁ = ‘ Z
-- same as x

t₂ : ∅ ⊢ X ⇒ X
```

```
t₂ = ƛ (‘ Z)
-- same as λx.x

t₃ : ∅ , X ⊢ X
t₃ = ƛ (‘ Z) · ‘ Z
-- same as (λx.x)y

t₄ : ∅ , X ⇒ X ⊢ X ⇒ X
t₄ = ƛ (‘ (S Z) · (‘ (S Z) · ‘ Z))
-- same as λx.y(yx)

t₅ : ∅ , X ⇒ X , X ⊢ X ⇒ X
t₅ = ƛ (‘ (S (S Z)) · (‘ (S (S Z)) · ‘ Z))
-- same as λx.y(yx), but with a different context
```

Now that we have defined types, contexts and terms in the standard $\lambda$-calculus, we define the $\lambda$-terms in the $\lambda_b$. Definition 5.13 gives the formal definition of $\lambda$-terms in the $\lambda_b$. Note that b is the suffix of many constructors, which is to distinguish between the constructors of the standard $\lambda$-calculus and $\lambda_b$.

**Definition 5.13. $\lambda_b$ in Agda.** This definition gives the data types for types, contexts and $\lambda$-terms of $\lambda_b$ in Agda. BOX A is a convenient abbreviation for Box (□ A). The definition of the $\lambda$-terms is based on the typing rules of $\lambda_b$, which are given in Definition 2.3.

```
data Typeb : Set
data BoxedTypeb : Set

data Typeb where
  _⇒b_ : BoxedTypeb → Typeb → Typeb
  X : Typeb
  Box : BoxedTypeb → Typeb

data BoxedTypeb where
  □ : Typeb → BoxedTypeb

BOX : Typeb → Typeb
BOX A = Box (□ A)

data Contextb : Set where
  ∅b : Contextb
  _,b_ : Contextb → BoxedTypeb → Contextb

data _∈b_ : BoxedTypeb → Contextb → Set where
  Zb : ∀ {Γ □A} → □A ∈b (Γ ,b □A)
```

```
  Sb : ∀ {Γ □A □B} → □A ∈b Γ → □A ∈b (Γ ,b □B)

data _⊢b_ : Contextb → Typeb → Set where
  ε : ∀ {Γ A}
    → □ A ∈b Γ
      -------
    → Γ ⊢b A

  box : ∀ {Γ A}
    → Γ ⊢b A
    --------------
    → Γ ⊢b BOX A

  ƛb : ∀ {Γ A B}
    → Γ ,b □ A ⊢b B
      ---------------
    → Γ ⊢b □ A ⇒b B

  _·b_ : ∀ {Γ A B}
    → Γ ⊢b □ A ⇒b B
    → Γ ⊢b BOX A
      ---------------
    → Γ ⊢b B
```

### 5.2.3 Substitution

The next challenge in the formalisation is a correct definition of substitution. This definition appeared to be the most complex part of the formalisation, and it caused the most problems with the proofs later in this thesis. The definition of substitution is similar to the definition that is used in the chapter "De Bruijn indices" in the PLFA book [27]. It is based on the definition by Conor McBride [6], which gives a similar definition of substitution in Rocq.

We only give the definition of substitution in Agda of $\lambda$-terms in $\lambda_b$, since the definition of substitution in the standard $\lambda$-calculus is exactly the same as the definition in [27]. Definition 5.15 gives the definition of parallel substitution, which is the substitution of a list of variables by a list of $\lambda$-terms. Then we define single substitution in Definition 5.16, which might be more familiar and is the substitution of a single $\lambda$-term by another $\lambda$-term.

In general, substitution is about replacing variables by $\lambda$-terms. With parallel substitution, a multiple variables are replaced by other $\lambda$-terms. So, substitution can be seen as a function, taking a map from variables in one context to $\lambda$-terms in another context and a $\lambda$-term as arguments, and returning a new $\lambda$-term with the variables replaced by the corresponding $\lambda$-terms.

For the definition of parallel substitution, we need to have an extension lemma, which is needed when we encounter an abstraction. An abstraction adds a new variable to the context, and we need a way to use the substitution function on the new context. This is where the extension lemma extsb comes in. It is a function from a map from variables in one context to $\lambda$-terms in another context, to a map from variables in the first context extended with a type to $\lambda$-terms in the second context extended with a type. The type that is used for extending the context is the input type of the $\lambda$ abstraction.

The extension lemma is defined in terms of renameb, which is used to construct a $\lambda$-term with an extended context. The renameb function takes a function from variables in one context to variables in another context, and returns a function from $\lambda$-terms in the first context to $\lambda$-terms in the second context. For example, renameb Sb $M$ renames $M$ by applying the function Sb to the variables in $M$. An example of this is given in Example 5.14.

The rename lemma is defined in terms of extb, which works similarly to extsb, but is only used for extending the context. The function extb takes a map from variables in one contex to variables in another context and returns a map from variables in the first context extended with a type to variables in the second context extended with a type. Again, this type will be the the input type of the $\lambda$ abstraction in renameb. Thus, the definition of simultaneous substitution is based on the definitions of extsb, renameb and extb.

**Example 5.14. Renaming in Agda.** Renaming $\mathsf{t}_1$ with Sb results in $\mathsf{t}_2$.

```
t₁ : ∅b ,b □ (□ X ⇒b X) ⊢b □ X ⇒b X
t₁ = ƛb (ε (Sb Zb) ·b box (ε (Sb Zb) ·b box (ε Zb)))

t₂ : ∅b ,b □ (□ X ⇒b X) ,b □ X ⊢b □ X ⇒b X
t₂ = ƛb (ε (Sb (Sb Zb)) ·b box (ε (Sb (Sb Zb)) ·b box (ε Zb)))


_ : renameb Sb t₁ ≡ t₂
_ = refl
```

**Definition 5.15. Parallel substitution in Agda of $\lambda_b$.** We define Parallel substitution in terms of extsb, renameb and extb.

```
extb : ∀ {Γ Δ}
  → (∀ {A} → A ∈b Γ → A ∈b Δ)
    ----------------------
  → (∀ {A B} → A ∈b Γ ,b B → A ∈b Δ ,b B)
extb f Zb = Zb
extb f (Sb x) = Sb (f x)
```

```
rename : ∀ {Γ Δ}
  → (∀ {A} → A ∈b Γ → A ∈b Δ)
    ----------------------
  → (∀ {A} → Γ ⊢b A → Δ ⊢b A)
rename f (ε x) = ε (f x)
rename f (box N) = box (rename f N)
rename f (ƛb M) = ƛb (rename (extb f) M)
rename f (M ·b N) = rename f M ·b (rename f N)

exts : ∀ {Γ Δ}
  → (∀ {A} → □ A ∈b Γ → Δ ⊢b A)
    ----------------------
  → (∀ {A B} → □ A ∈b Γ ,b B → Δ ,b B ⊢b A)
exts f Zb = ε Zb
exts f (Sb x) = rename Sb ( f x )

substb : ∀ {Γ Δ}
  → (∀ {A} → □ A ∈b Γ → Δ ⊢b A)
    ------------------------
  → (∀ {A} → Γ ⊢b A → Δ ⊢b A)
substb f (ε x) = f x
substb f (box N) = box (substb f N)
substb f (ƛb M) = ƛb (substb (exts f) M)
substb f (M ·b N) = substb f M ·b (substb f N)
```

Single substitution can be defined by using parallel substitution. The map fb in Definition 5.16 is a function that takes a variable in the context and returns a $\lambda$-term. It only returns the $\lambda$-term that is substituted if the variable is the most recent variable in the context. Single substitution of the $\lambda$-term that occurs most recently in the context of $M$ by $\lambda$-term $N$ is denoted by $M$ [ $N$ ]b in the formalisation of $\lambda_b$.

**Definition 5.16. Single substitution in Agda of $\lambda_b$.** Single substitution is defined in terms of parallel substitution, using the function fb that replaces the most recent variable in the context with the $\lambda$-term that is substituted.

```
fb : ∀ {Γ C} (M : Γ ⊢b C) → ∀ {A} → □ A ∈b Γ ,b □ C → Γ ⊢b A
fb M Zb = M
fb M (Sb x) = ε x

_[_]b : ∀ {Γ A C}
  → Γ ,b □ C ⊢b A
  → Γ ⊢b C
    -------------
  → Γ ⊢b A
_[_]b N M = substb (fb M) N
```

Example 5.17 gives a demonstration of single substitution in $\lambda_\flat$.

**Example 5.17. Single substitution in Agda of $\lambda_\flat$.** This example shows that $\mathsf{t}_1\ [\ \mathsf{t}_2\ ]\mathsf{b} \equiv\ \mathsf{t}_3$

```
t₁ : ∅b ,b □ (□ X ⇒b X) ⊢b □ X ⇒b X
t₁ = ƛb (ε (Sb Zb) ·b box (ε (Sb Zb) ·b box (ε Zb)))

t₂ : ∅b ⊢b □ X ⇒b X
t₂ = ƛb (ε Zb)

t₃ : ∅b ⊢b □ X ⇒b X
t₃ = ƛb (ƛb (ε Zb) ·b box (ƛb (ε Zb) ·b box (ε Zb)))

_ : t₁ [ t₂ ]b ≡ t₃
_ = refl
```

### 5.2.4 Weakening and Renaming

Weakening and renaming becomes something to take into consideration in the formalisation. Two $\lambda$-terms that are the same except for the context are not exactly the same. An example to demonstrate this is the formal definition of raise in Agda, where the use of renameb is required. Recall that raise was defined as follows:

$$\mathsf{raise}(M) := \lambda z.\varepsilon(z)M$$

The term $M$ outside the function (in $\mathsf{raise}(M)$) has a different context than the term $M$ inside the function of raise ($\lambda z.\varepsilon(z)M$). We attempt to give a typing derivation in Figure 5.1. We assume that term $M$ has type B' which can be derived from $\Gamma$. However, we are stuck in the derivation since we do not have rules to discard assumptions. We see that we need to proof $\Gamma, z : \Box(\Box B \to B') \vdash M : B'$ instead of $\Gamma \vdash M : B'$.

$$\dfrac{\dfrac{\dfrac{}{\Gamma, z : \Box(\Box B \to B') \vdash \varepsilon(z) : \Box B \to B'}\,\mathrm{Id}_\Box \qquad \begin{array}{c}\vdots\\ \Gamma, z : \Box(\Box B \to B') \vdash M : B'\end{array}}{\Gamma, z : \Box(\Box B \to B') \vdash \varepsilon(z)M : B'}\,{\to}\text{-E}}{\Gamma \vdash \lambda z.\varepsilon(z)M : \Box(\Box B \to B') \to B'}\,{\to}\text{-I}$$

Figure 5.1: The attempt of the typing derivation of $\lambda z.\varepsilon(z)M$

Though we cannot immediately conclude following below, it does make sense that this is the case.

$$\Gamma \vdash M : B' \text{ implies } \Gamma, z : \Box(\Box B \to \Box B') \vdash M : \Box B'$$

We only add an extra assumption that we do not use, so we need a function that can handle this. This is where renameb comes in. The function renameb "renames" the de Bruijn indices in the $\lambda$-terms with a certain function and changes the context accordingly. In our case, we only need to rename the indices with Sb (S for $\lambda_b$).

Let us show how this works in the formalisation. Assume that we have a $\lambda$-term $M$ that has type $B'$ in context $\Gamma$, which is denoted by $M : \Gamma \vdash B'$ in the formalisation. We use renameb in such a way that it returns a $\lambda$-term of the type $\Gamma\ ,\mathsf{b}\ \Box(\Box B \to B') \vdash \Box B'$. We achieve this by renaming with the function Sb as shown below. The resulting type is $\Gamma\ ,\mathsf{b}\ \Box(\Box B \to \Box B') \vdash \Box B'$

$$\mathsf{rename}\ (\mathsf{Sb}\ \{\Box B = \Box(\Box B \to \Box B')\})\ M$$

Here, we need to give the implicit argument since Sb needs to know what to the resulting context will be. Now, we incorporate this into the definition of raise. Note that Agda can automatically infer the implicit arguments of Sb in the definition of raise.

**Definition 5.18. raise** in Agda

```
raise : ∀ {Γ A B}
  → (t : Γ ⊢b BOX B)
  → Γ ⊢b □ (□ B ⇒b BOX A) ⇒b BOX A
raise t = ƛb (ε Zb ·b renameb Sb t)
```

## 5.3 Overview of the Formalisation

This section provides an overview of the formalisation in Agda. We show how propositions in section 4.1 and section 4.2 are formalised in Agda. We also give the main helper lemmas that are needed to prove these propositions. The formalisation is available on GitHub[1]

### 5.3.1 File Structure

The Agda formalisation is structured into a number of files. Here, we go over the file structure. The project has the following three main directories: `examples`, `formalisation` and `proofs`. The `examples` directory contains some examples of $\lambda$-terms, substitution, embeddings and reductions. The `formalisation` directory is the most important directory and contains the formalisation of $\lambda$-terms, substitution, embeddings and reductions. Here, we list all files in this directory and give a short description of their contents:

[1] `www.thisurlwillbeavailablelater.com`

- `boxCalc.agda`: Contains the definition of $\lambda$-terms in $\lambda_b$, the definition of substitution and the definition of values in $\lambda_b$.
- `boxCalcRed.agda`: Contains all the reduction relations in $\lambda_b$.
- `calc.agda`: Contains the definition of $\lambda$-terms in the standard $\lambda$-calculus, the definition of substitution and the definition of values.
- `embedCBNIntoCBB.agda`: Contains the definition of Girard's embedding.
- `embedCBVIntoCBB.agda`: Contains the definition of Gödel's embedding.
- `nameCalcRed.agda`: Contains the reduction relations for $\lambda_n$.
- `valueCalcRed.agda`: Contains the reduction relations for $\lambda_v$.

The `proofs` directory contains the proofs of the propositions in section 4.1 and section 4.2 in the files `nameProofs.agda` and `valueProofs.agda` respectively. Both files also contain all helper lemmas necessary to prove the propositions.

### 5.3.2 Helper Lemmas

This subsection goes over the two main helper lemmas needed to prove the main propositions about Girard's and Gödel's embedding. In general, the lemma pres-subst states that the order of embedding and substitution does not matter. Lemma 5.19 shows the corresponding Agda proof of this lemma for Girard's embedding. We do not give the full proof, since it is quite long and involves going through the chained definition of substitution, traversing extsb, renameb and extb. Lemma 5.20 shows the proof of pres-subst for Gödel's embedding. This lemma consists of two parts, one for substitution in values and one for substitution in $\lambda$-terms in general.

**Lemma 5.19. Preservation of single substitution in Girard's embedding.** This lemma is defined in `proofs/nameProofs.agda`

```
-- Preservation and Reflection of
-- single substitution of Girard's translation
pres-subst : ∀ {Γ A B}
  → (M : Γ , B ⊢ A)
  → (N : Γ ⊢ B)
  → (embedTerm M) [ embedTerm N ]b ≡ embedTerm (M [ N ])
pres-subst M N =
  pres-simultaneous-subst
    (fb (embedTerm N))
    (f N)
    (pres-subst-func N) M
```

**Lemma 5.20. Preservation of single substitution in Gödel's embedding.** This lemma is defined in `proofs/valueProofs.agda`

```
-- Preservation and Reflection of
-- single substitution of Gödel's translation
pres-subst : ∀ {Γ A B}
  → (M : Γ , B ⊢ A)
  → (V : Γ ⊢ B)
  → (x : Value V)
  → embedTerm M [ embedValue V x ]b ≡ embedTerm (M [ V ])
pres-subst M V x =
  pres-simultaneous-subst
    (fb (embedValue V x))
    (f V)
    (pres-subst-func V x)
    M


-- Preservation and Reflection of
-- single substitution in values of Gödel's translation
pres-subst-val : ∀ {Γ A B}
  → (M : Γ , B ⊢ A)
  → (V : Γ ⊢ B)
  → (x : Value M)
  → (y : Value V)
  → embedValue M x [ embedValue V y ]b ≡ embedValue (M [ V ]) (val-after-subst M V x y)
pres-subst-val (` Z) V x y = refl
pres-subst-val (` (S x₁)) V x y = refl
pres-subst-val (ƛ M) V x y =
  beginEq
    embedValue (ƛ M) x [ embedValue V y ]b
  ≡⟨ refl ⟩
    ƛb (substb (extsb (fb (embedValue V y))) (embedTerm M))
  ≡⟨
    cong
      ƛb
      (pres-simultaneous-subst
        (extsb (fb (embedValue V y)))
        (exts (f V))
        (exts-pres-subst-func V y)
        M)
    ⟩
    ƛb (embedTerm (subst (exts (f V)) M))
  ≡⟨ refl ⟩
    embedValue ((ƛ M) [ V ]) (val-after-subst (ƛ M) V x y)
  □eq
```

Due to the use of renameb in the definition of raise in Gödel's embedding, we also needed many other helper lemmas such as subst-renameb-comp, substb-rename-commute and substb-cong. Since subst is defined by extsb, which is defined by renameb and extb, we also need composition, commutativity and congruence lemmas for extsb, renameb and extb. However, these lemmas are not that interesting for this thesis, and we do not give them here. These lemmas can be found in the Agda formalisation in `proofs/valueProofs.agda`.

### 5.3.3 Proofs of the Main Propositions

This subsection gives proofs of the propositions about Girard's and Gödel's embedding. We start with the propositions about Girard's embedding, and then we continue with the propositions about Gödel's embedding. The first proposition, proposition 4.4, states that Girard's embedding preserves reduction. That is, if we have a $\lambda$-term $M$ and we reduce it to $N$ in $\lambda_n$, then the translation of $M$ under Girard's embedding also reduces to the translation of $N$ under Girard's embedding. We repeat the proposition here for convenience:

For all $\lambda$-term $M$ and $N$, if $M \to_{\beta_n} N$ in $\lambda_n$, then $M^\circ \to_{\beta_b} N^\circ$.

Proof 5.21 shows the Agda proof of this proposition. Note that we use the helper lemma lemma 4.3 to prove this proposition.

**Proof 5.21. Preservation of reduction under Girard's translation.**

```
-- Preservation of reduction of Girard's translation
pres-red : ∀ {Γ A}
  → (M : Γ ⊢ A)
  → (N : Γ ⊢ A)
  → (M ⇝βn N)
  → (embedTerm M ⇝βb embedTerm N)
pres-red (ƛ M₁ · M₂) N βn =
  Eq.subst (λ t → ƛb (embedTerm M₁) ·b box (embedTerm M₂) ⇝βb t)
  (pres-subst M₁ M₂)
  βb
pres-red M N (μ {M = M₁} {M' = M'} red) =
  μ (pres-red M₁ M' red)
pres-red M N (ν¡ {M = M₁} {N = N₁} {N' = N'} x red) =
  ν (ζ (pres-red N₁ N' red))
pres-red M N (ν {M = M₁} {N = N₁} {N' = N'} red) =
  ν (ζ (pres-red N₁ N' red))
pres-red M N (ξ {M = M₁} {M' = M'} red) =
  ξ (pres-red M₁ M' red)
```

□

The second proposition about Girard's translation is proposition 4.5. It states that if a $\lambda$-term $M$ evaluates to $N$ in $\lambda_n$ with the relation $\to_n$, then the translation of $M$ under Girard's embedding also evaluates to the translation of $N$ under Girard's embedding with $\to_{b_n}$. Again, we repeat the proposition here for convenience:

For all $\lambda$-term $M$ and $N$, if $M \to_n N$ in $\lambda_n$, then $M^\circ \to_{b_n} N^\circ$.

Proof 5.22 shows the Agda proof of this proposition. The proof is similar to proof 5.21. The only difference is that we have less cases for the evaluation relation $\to_n$ and thus the proof is shorter.

**Proof 5.22. Preservation of evaluation under Girard's translation.**

```
-- Preservation of evaluation of Girard's translation
pres-eval : ∀ {Γ A}
  → (M : Γ ⊢ A)
  → (N : Γ ⊢ A)
  → (M ⇝n N)
  → (embedTerm M ⇝bn embedTerm N)
pres-eval (ƛ M₁ · M₂) N βn =
  Eq.subst
    (λ t → ƛb (embedTerm M₁) ·b box (embedTerm M₂) ⇝bn t)
    (pres-subst M₁ M₂)
    βb
pres-eval M N (μ {M = M₁} {M' = M'} red) =
  μ (pres-eval M₁ M' red)
```

□

The next two propositions are about Gödel's embedding. The first proposition of Gödel's embedding we proved is proposition 4.9. It states if a $\lambda$-term $M$ in $\lambda_v$ reduces to $N$ in one step, then $M^*$ reduces to $N^*$ in $\lambda_b$ in two steps. We repeat the proposition here for convenience:

For all $\lambda$-terms $M$ and $N$, if $M \to_{\beta_n} N$ in $\lambda_v$, then $M^* \to^2_{\beta_b} N^*$.

Proof 5.24 shows the Agda proof of this proposition. Note that it makes use of the helper lemma pres-rename-red-2, which states that if a $\lambda$-term $M$ reduces to $N$ in two steps, then renameb $fb'$ $M$ also reduces to renameb $fb'$ $N$ in two steps. We do not go into detail about this lemma, but it can be found in `proofs/valueProofs.agda`.

**Proof 5.23. Preservation of reduction** under Gödel's translation with $\to^2_{\beta_b}$ reduction rule.

```
-- Preservation of 2 step reduction of Gödel's translation
pres-red : ∀ {Γ A}
  → (M : Γ ⊢ A)
  → (N : Γ ⊢ A)
  → (M ⇝βv N)
  → (embedTerm M ⇝βb² embedTerm N)
pres-red (ƛ M₁ · M₂) N (βv x) =
  βb²
  (raise (embedTerm M₂) ·b box (ƛb (embedTerm M₁)))
  (ƛb (embedTerm M₁) ·b box (embedValue M₂ x))
  (embedTerm (M₁ [ M₂ ]))
  (Eq.subst
    (λ x →
      ƛb (ε Zb ·b renameb Sb (embedTerm M₂)) ·b box (ƛb (embedTerm M₁))
      ⇝βb
      x)
```

```
    (cong₂ _·b_ refl (subst-rename-fb-comp M₁ M₂ x))
    βb)
  (Eq.subst
    (λ t → ƛb (embedTerm M₁) ·b box (embedValue M₂ x) ⇝βb t)
    (pres-subst M₁ M₂ x)
    βb)
pres-red M N (μ {M = M₁} {M' = M'} red) =
  ν (pres-red M₁ M' red)
pres-red M N (ν¡ {M = M₁} {N = N₁} {N' = N'} x red) =
  μ (ξ (ν (pres-rename-red-2
            Sb
            (embedTerm N₁)
            (embedTerm N')
            (pres-red N₁ N' red))))
pres-red M N (ν {M = M₁} {N = N₁} {N' = N'} red) =
  μ (ξ (ν (pres-rename-red-2
            Sb
            (embedTerm N₁)
            (embedTerm N')
            (pres-red N₁ N' red))))
pres-red M N (ξ {M = M₁} {M' = M'} red) =
  ζ (ξ (pres-red M₁ M' red))
```

□

The last proposition we proved is proposition 4.9. It states that if $M \to_{\beta_v} N$ in $\lambda_v$, then $M^* \to_{\beta_{b2}} N^*$. We repeat the proposition here for convenience:

For all $\lambda$-terms $M$ and $N$, if $M \to_{\beta_v} N$ in $\lambda_n$, then $M^* \to_{\beta_{b2}} N^*$.

The proof of this proposition is similar to proof 5.24 and is given in proof 5.24. We use the helper lemma pres-rename-red- $\rightsquigarrow \beta_{b2}$ that is similar to pres-rename-red-2 but for the $\to_{\beta_{b2}}$ reduction rule. The proof of this lemma can be found in `proofs/valueProofs.agda`.

**Proof 5.24. Preservation of reduction** under Gödel's translation with $\to_{\beta_{b2}}$ reduction rule.

```
-- Preservation of reduction by Gödel's translation
-- with the two step reduction rule
pres-red⇝βb₂ : ∀ {Γ A}
  → (M : Γ ⊢ A)
  → (N : Γ ⊢ A)
  → (M ⇝βᵥ N)
  → (embedTerm M ⇝βb₂ embedTerm N)
pres-red⇝βb₂ (ƛ M₁ · M₂) N (βᵥ x) =
  Eq.subst
    (λ t → t ⇝βb₂ embedTerm (M₁ [ M₂ ]))
    (cong₂
      _·b_
```

```
      (cong
        ƛb
        (cong₂ _·b_ refl (cong (renameb Sb) (box-embedValue x))))
      refl)
    (Eq.subst
      (λ t →
        raise ( box (embedValue M₂ x) ) ·b box (ƛb (embedTerm M₁)) ⇝βb₂ t)
      (pres-subst M₁ M₂ x)
      βb₂
    )
pres-red⇝βb₂ M N (μ {M = M₁} {M' = M'} red) =
  ν (pres-red⇝βb₂ M₁ M' red)
pres-red⇝βb₂ M N (ν¡ {N = N₁} {N' = N'} x red) =
  μ (ξ (ν (pres-rename-red-⇝βb₂
             Sb
             (embedTerm N₁)
             (embedTerm N')
             (pres-red⇝βb₂ N₁ N' red))))
pres-red⇝βb₂ M N (ν {N = N₁} {N' = N'} red) =
  μ (ξ (ν (pres-rename-red-⇝βb₂
             Sb
             (embedTerm N₁)
             (embedTerm N')
             (pres-red⇝βb₂ N₁ N' red))))
pres-red⇝βb₂ M N (ξ {M = M₁} {M' = M'} red) =
  ζ (ξ (pres-red⇝βb₂ M₁ M' red))
```

□

# Chapter 6

# Related Work

In this chapter you demonstrate that you are sufficiently aware of the state-of-art knowledge of the problem domain that you have investigated as well as demonstrating that you have found a *new* solution / approach / method.

# Chapter 7

# Conclusions and Future Work

## 7.1 Conclusions

This thesis has formalised the call-by-box lambda calculus, a refined modal $\lambda$-calculus introduced by [23]. We implemented its syntax, substitution, reduction and evaluation relations of $\lambda_b$ in Agda. We also formalised Girard's and Gödel's embeddings which are embeddings from the standard call-by-name $\lambda$-calculus ($\lambda_n$) and call-by-value $\lambda$-calculus ($\lambda_v$) to the shared target, the call-by-box $\lambda$-calculus ($\lambda_b$). For both embeddings, we have proven some properties that are preserved in $\lambda_b$. We have proven the following propositions formally in Agda:

- Girard's embedding preserves reduction (proposition 4.4).
- Girard's embedding preserves evaluation (proposition 4.5).
- Gödel's embedding preserves reduction (proposition 4.9 and 4.11).

In order to prove these propositions, we made use of several helper lemmas. The most important helper lemma is lemma 4.3 for the properties about Girard's translation, and lemma 4.8 for the properties about Gödel's translation. These helper lemmas state that substitution is preserved and reflected by the translation. They were the most complex to prove, but they are fundamental for the proofs about preservation of reduction and evaluation.

We also saw that the formal definition of raise in Gödel's embedding is more complex than less formal definition due to the use of the rename function. This rename function made proofs about Gödel's embedding significantly more complex than those about Girard's embedding, which is one of the reasons why we did not formally prove preservation of evaluation for Gödel's embedding.

In conclusion, this thesis provides a formal foundation for $\lambda_b$ in Agda. It contains the syntax, substitution, reduction and evaluation relations as well as some proofs about Girard's and Gödel's embeddings. The formalisation could serve as a basis for further research into the unification of call-by-name and call-by-value evaluation strategies in a modal setting, and could also serve as a foundation for future programming languages or proof assistants that natively support both evaluation strategies.

## 7.2 Future Work

This thesis has formalised the core functionalities of $\lambda_b$ and has proved some important properties of Girard's and Gödel's embeddings. However, there are still many areas for future work regarding this formalisation. We list some opportunities for future work below.

- Our formalisation of $\lambda_b$ can be extended with more features, such as case expressions, natural numbers or recursion. The book PLFA [27] implements these features in the standard $\lambda$-calculus, and they can be implemented in $\lambda_b$ as well.
- Preservation of evaluation under Gödel's embedding has not been proven formally yet, but it is an interesting property to prove. It would make the formalisation more complete and could be useful for future research.
- We can extend the formalisation with more properties, such as reflection of reduction and evaluation under Girard's and Gödel's embeddings. This would show that the target $\lambda_b$, does not simulate any other reductions or evaluations than those of $\lambda_n$ and $\lambda_v$. For example, reflection of reduction under Girard's translation can be stated as follows: if $M^\circ \rightarrow_{\beta_b} N^\circ$, then $M \rightarrow_{\beta_n} N$ in $\lambda_n$.
- We only formalised $\lambda_b$ in Agda, but it could be interesting to also formalise different framworks that can simulate both cbn and cbv, such as the linear logic approach by Thomas Ehrhard and Giulio Guerrieri [13] or Maraist et al. [19], and the thunk-based approach by Hatcliff, John and Danny, Olivier [15]. All these formalisations could then be compared to see which is simpler, more efficient or more powerful.
- The boxed $\lambda$-calculus can also be used to construct an interpreter for a programming language that supports both cbn and cbv evaluation. This is a more practical application and it could be interesting to see $\lambda_b$ in practice.

# Bibliography


[1] Agda emacs mode. `https://agda.readthedocs.io/en/v2.6.4/tools/emacs-mode.html`. Accessed: 2025-06-06.

[2] Agda documentation. `https://agda.readthedocs.io/`, 2023. Accessed: 2025-06-14.

[3] Thorsten Altenkirch and Bernhard Reus. Monadic presentations of lambda terms using generalized inductive types. In Jörg Flum and Mario Rodriguez-Artalejo, editors, *Computer Science Logic*, pages 453–468, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

[4] Zena M. Ariola and Matthias Felleisen. The call-by-need lambda calculus. *Journal of Functional Programming*, 8(3):265–301, 1997.

[5] Henk P. Barendregt. Lambda calculi with types. In S. Abramsky, Dov M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 2, pages 117–309. Oxford University Press, Oxford, 1992.

[6] Nick Benton, Chung-Kil Hur, Andrew J. Kennedy, and Conor McBride. Strongly typed term representations in coq. *Journal of Automated Reasoning*, 49(2):141–159, 2012.

[7] G. M. Bierman and Valeria de Paiva. On an intuitionistic modal logic. *Studia Logica*, 65(3):383–416, 2000.

[8] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936.

[9] Alonzo Church. A formulation of the simple theory of types. *The Journal of Symbolic Logic*, 5(2):56–68, 1940.

[10] Haskell B. Curry and Robert Feys. *Combinatory Logic, Volume I.* North-Holland Publishing Company, 1958.

[11] Haskell B. Curry, Robert Feys, and William Craig. *Combinatory Logic, Volume I.* North-Holland Publishing Company, 1958. Introduces substitution notation $[a/x]b$.

[12] N.G de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the church-rosser theorem. *Indagationes Mathematicae (Proceedings)*, 75(5):381–392, 1972.

[13] Thomas Ehrhard and Giulio Guerrieri. The bang calculus: An untyped lambda-calculus generalizing call-by-name and call-by-value. In *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming (PPDP 2016)*, pages 174–187. ACM, 2016.

[14] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987.

[15] JOHN HATCLIFF and OLIVIER DANVY. Thunks and the $\lambda$-calculus. *Journal of Functional Programming*, 7(3):303–319, 1997.

[16] Arend Heyting. Die formalen regeln der intuitionistischen logik. *Sitzungsberichte der Preußischen Akademie der Wissenschaften*, pages 42–56, 1930.

[17] J. Roger Hindley and Jonathan P. Seldin. *Lambda-Calculus and Combinators: An Introduction.* Cambridge University Press, 2nd edition, 2008.

[18] William A. Howard. The formulae-as-types notion of construction. In J. P. Seldin and J. R. Hindley, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 479–490. Academic Press, 1980.

[19] J. Maraist, M. Odersky, D.N. Turner, and P. Wadler. Call-by-name, call-by-value, call-by-need and the linear lambda calculus. *Theoretical Computer Science*, 228(1):175–210, 1999.

[20] J. C. C. McKinsey and Alfred Tarski. Some theorems about the sentential calculi of lewis and heyting. *Journal of Symbolic Logic*, 13(1):1–15, 1948.

[21] Ulf Norell. *Towards a Practical Programming Language Based on Dependent Type Theory.* PhD thesis, Chalmers University of Technology, 2007.

[22] G.D. Plotkin. Call-by-name, call-by-value and the $\lambda$-calculus. *Theoretical Computer Science*, 1(2):125–159, 1975.

[23] José Espírito Santo, Luís Pinto, and Tarmo Uustalu. Modal Embeddings and Calling Paradigms. In Herman Geuvers, editor, *4th International Conference on Formal Structures for Computation and Deduction*

*(FSCD 2019)*, volume 131 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:20, Dagstuhl, Germany, 2019. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.

[24] A. S. Troelstra. Introductory note to 1933f. In Solomon Feferman, John W. Dawson Jr., Stephen C. Kleene, Gregory H. Moore, Robert M. Solovay, and Jean van Heijenoort, editors, *Collected Works, Volume I: Publications 1929–1936*, pages 296–299. Oxford University Press, New York, 1986.

[25] A. S. Troelstra. *Lectures on Linear Logic*. CSLI Publications, Stanford, CA, 1992.

[26] A. S. Troelstra and D. van Dalen. *Constructivism in Mathematics: An Introduction, Volume I*. North-Holland, Amsterdam, 1988.

[27] Philip Wadler, Wen Kokke, and Jeremy G. Siek. *De Bruijn Indices*. University of Edinburgh, August 2022. Version 22.08, accessed 2025-06-06.

# Appendix A

# Appendix

Appendices are *optional* chapters in which you cover additional material that is required to support your hypothesis, experiments, measurements, conclusions, etc. that would otherwise clutter the presentation of your research.